DIGITALAIKART

THE 2026-27 INDIA CAREER PLAYBOOK

THE GCC FRESHER PLAYBOOK

How to get hired by a Global Capability Centre in India

30 CHAPTERS - 7 PARTS

2,117 GCCs | $98.4 bn revenue | 2.36 mn professionals

The complete route into India's fastest-growing employer segment: lane selection, proof of work, referrals, greenfield GCCs, hackathons, the interview loop, internship conversion, and the 90-day plan.

VERIFIED AGAINST ZINNOV-NASSCOM, TEAMLEASE, EY, FOUNDIT AND GOVERNMENT OF INDIA SOURCES

OCTOBER 2026 | DIGITAL EDITION | DIGITALKARTAI.SHOP

Contents

What a GCC actually is . . . 7

The FY26 numbers, and why they matter to you . . . 9

The fresher squeeze, explained properly . . . 11

Four kinds of GCC, and which ones hire freshers . . . 13

The seven entry lanes . . . 16

Reading a GCC job description like a recruiter . . . 20

Pay: what is real, what is noise . . . 22

The city arbitrage . . . 25

The one-project rule . . . 28

Five project blueprints . . . 30

SQL to the level they actually test . . . 33

The AI-adjacent skills that get you shortlisted . . . 35

GitHub and portfolio hygiene . . . 37

LinkedIn is a search index, not a CV . . . 40

The referral playbook . . . 42

The greenfield strategy . . . 45

Programmes and hackathons: the real calendar . . . 47

Careers pages and ATS: applying so a human sees it . . . 50

The loop, round by round . . . 53

The online assessment . . . 55

The project deep-dive: six questions you will always get . . . 57

Core computer science at GCC depth . . . 59

The behavioural round: eight stories to prepare 61

The ten-minute explanation test 63

Internship to offer: the conversion playbook 65

Reading the offer 67

Scam defence 69

The 90-day plan 72

Building your 30-company target list 74

The worksheets 76

How to use this book

This is not a motivational book. It is a set of instructions, written for one specific situation: you have a degree, or you are about to get one, and you want a job at a Global Capability Centre in India. Not a generic software job. Not a mass-hiring trainee seat. A seat inside the India office of a global company, where you work on that company's own products and systems.

That goal is achievable, and it is unusually well documented. The catch is that almost nobody explains it properly, because the path into a GCC does not look like the path into an IT services company. There is no single national test. There is no 10,000-offer campus drive. The pipeline is smaller, it starts earlier, and it is heavily weighted towards people who can prove something rather than people who can recite something.

So this book is organised in the order you will actually need it:

- **Part One** explains the market - what a GCC is, how big the sector is, and exactly how fresher hiring has changed in the last three years. Read this even if you think you already know, because most of what circulates on LinkedIn about this market is out of date or wrong.
- **Part Two** helps you pick a lane: which of the seven entry routes suits you, how to read a job description for what it really wants, what the money actually is, and which city to target.
- **Part Three** is the part that decides whether you get shortlisted: proof of work. One project, built properly, beats five certificates.
- **Part Four** is distribution. How to be findable, how referrals actually work, how to catch programmes before they close, and how to apply so a human sees your application.
- **Part Five** is the interview loop, round by round, with the questions you will actually be asked.
- **Part Six** is conversion: turning an internship into an offer, reading an offer letter properly, and defending yourself against the scams that cluster around this market.
- **Part Seven** is a 90-day execution plan with worksheets you can fill in.

If you only have one hour, read the next two pages, then jump to Chapter 9 and Chapter 14. Those three pieces - the market map, the project, and being findable - carry most of the result.

> **A NOTE ON DATES AND VERIFICATION**
>
> Every number in this book is sourced and dated in the appendix. Where a figure comes from an industry survey rather than an official government release, it is labelled as such. Where a programme date or eligibility rule is involved, you are told to verify it on the organiser's own page before you spend a single hour on it. Programmes move their windows every cycle. This book teaches you the shape of the process; the organiser's page is the only authority on the current dates.

The whole book on one page

If you read nothing else, read this. Everything after it is detail, evidence and scripts.

1. **The market moved.** IT services cut fresher hiring by about 11 per cent in FY26, from roughly 125,000 to 110,000. GCCs hired roughly 140,000 to 150,000 freshers in the same year and are expected to grow that by about 12 per cent in FY27. The door did not close. It moved to a different building.
2. **The pay gap is now structural.** Industry surveys put GCC entry-level pay at roughly Rs 7.5 lakh a year against about Rs 4.5 lakh in IT services, and the gap has been widening for three years. Same degree, same city, different employer type.
3. **GCCs do not run mass drives.** There is no NQT equivalent. Entry happens through careers pages, graduate programmes, hackathons, referrals, internships and - increasingly - recruiters finding you on LinkedIn. This is the single biggest reason capable people fail to get in.
4. **Skills-first is real, not a slogan.** Around 15 per cent of fresher postings no longer specify an educational qualification at all. What they do specify is a stack, a project and the ability to explain it.
5. **One project, deeply understood, beats five shallow ones.** Every interview will spend more time on the one thing you claim to have built than on anything else. Build one thing you can defend for forty minutes.

6 **Internships are the highest-conversion door.** Industry estimates put internship-to-offer conversion at 40 to 70 per cent at leading GCCs. If you get an internship, treat it as a three-month interview, because that is what it is.

7 **Tier-2 cities are the arbitrage.** Emerging-city GCC hiring is growing roughly 23 to 42 per cent year on year, against high-teens in metros, on a much smaller applicant pool. Coimbatore, Jaipur, Indore, Kochi, Bhubaneswar, Visakhapatnam.

8 **Communication is screened in every round, not just HR.** Roughly a third of tech job descriptions name communication as a requirement - as often as Python. GCCs work with teams in other time zones. If you cannot explain your work to a stranger in ten minutes, that is the skill to fix first.

9 **Apply through the front door, early.** Careers page plus a referral, inside the first few days of a posting, beats a hundred late aggregator clicks.

10 **Run the 90-day plan.** Chapter 28. Weeks 1 to 4 target list and profile, weeks 5 to 8 proof of work, weeks 9 to 12 applications and interview sprint. Doing this in order is most of the advantage.

PART ONE

The Market You Are Actually Walking Into

Before tactics, the terrain. Four chapters on what a GCC is, how large the sector has become, why fresher hiring contracted in one place and expanded in another, and which kinds of GCC actually hire people with no experience. If you skip this, you will optimise for the wrong employer.

CHAPTER 1

What a GCC actually is

A Global Capability Centre is a company's own office in another country. It is not an outsourcing vendor. It is not a business process outsourcer taking work on contract. It is the parent company, in another geography, on the parent company's own payroll, building and running the parent company's own systems.

When JPMorgan, Walmart, Goldman Sachs, Shell, Airbus, Siemens, Target, Tesco, AstraZeneca or HSBC sets up a large technology and operations centre in Bengaluru, Hyderabad, Pune or Chennai, that centre is a GCC. Sometimes you will see it called a captive centre, a global in-house centre, or a capability centre. The older term was simply 'captive'. The industry settled on Global Capability Centre around 2021, and the acronym stuck.

The distinction that matters on your payslip

There are three employer types a fresh graduate in India typically compares, and they behave very differently. It is worth being precise about this, because most career advice blurs them together.

	IT services company	GCC	Product / startup
Who pays you	A vendor selling engineering hours to clients	The parent company directly	The company that owns the product
What you work on	Client projects; you move between accounts	The parent's own products and platforms	One product, usually
Revenue logic	Billable utilisation. Margins around 19-25 per cent.	Cost centre funded by global parent	Growth, then margin
Hiring shape	High volume, structured, campus-heavy	Smaller volume, selective, skills-first	Very selective, referral-heavy
Fresher intake	Large but shrinking	Growing	Small
Typical fresher pay	Rs 3.5-4.5 lakh	Rs 6-9 lakh, more in niche tracks	Rs 8-14 lakh, high variance
Onsite travel	Common and early	Rare by design	Rare

Two rows in that table explain almost everything else. IT services earns margin by billing hours, so it hires in bulk, trains in bulk, and keeps utilisation high. A GCC is funded by a global parent and exists to build capability, so it hires fewer people and pays more for the ones it wants. The pay difference is not generosity. It is a different business model.

> **THE SENTENCE TO REMEMBER**
> An IT services company sells your hours. A GCC buys your capability. That single difference is why the interview, the pay and the hiring channel all look different.

What a GCC is not

- **It is not a call centre.** Some GCCs contain support functions, but the sector's centre of gravity has moved. In the FY2026 landscape, product management roles inside Indian GCCs have nearly doubled to about 20 per cent of the mix from 11.4 per cent five years earlier, and niche-skill roles have grown from 20 per cent to 27 per cent.

- **It is not second-tier work.** In the EY GCC Pulse Survey 2025, 92 per cent of GCC leaders said their centres contribute well beyond cost arbitrage, and 87 per cent said they own end-to-end global processes. That means the India team owns the process, not a slice of it.
- **It is not the same as a startup.** You get enterprise structure, defined bands, benefits and stability - and in exchange, slower title progression and less product variety. Chapter 26 covers what that trade actually costs you.
- **It is not a guaranteed visa route abroad.** Broad onsite deployment is uncommon at GCCs by design, because the centre exists so the work happens in India. If early international travel is your priority, that is a genuine argument for IT services instead.

The awkward part nobody puts in the brochure

GCC hierarchies are flat. A title change can take three to four years rather than two. You will usually work on one product or one business unit for a long stretch, which builds depth but narrows breadth. And when a global parent restructures, the India centre is not insulated - it feels the decision quickly.

None of that is a reason to avoid GCCs. It is a reason to go in with accurate expectations, so that in year three you are not surprised. The people who are unhappy at GCCs are usually the ones who joined expecting a startup and got an enterprise, or joined expecting a fast ladder and got a wide one.

CHAPTER 2

The FY26 numbers, and why they matter to you

Here is where the sector actually stands. These are the figures from the Zinnov-NASSCOM India GCC Landscape Report for FY2026, published in May 2026, with data as of March 2026.

2,117	$98.4 bn	2.36 mn	3,728
GCCS	REVENUE	PEOPLE	UNITS
operating in India	annual, FY26	employed directly	across those centres

Read those four numbers again with a specific question in mind: how did this get so large, so fast? The sector has grown 32 per cent in the number of GCCs since FY2021 alone, and revenue has moved from roughly $65 billion two years ago to $98.4 billion. A target that NASSCOM had set for 2030 was effectively reached in FY26, four years early.

The shape of the sector

Segment	Count	What it means
Forbes Global 2000 companies	506	The largest global enterprises, already here
Mid-market GCCs	583	Revenue between $100 mn and $1 bn
PE-backed GCCs	504	Private-equity-owned groups, expanding fastest
Below $100 mn revenue	423	Smaller or newer centres

The mid-market and PE-backed segments matter disproportionately to a fresher. A PE-backed group under a three-year value-creation plan is under pressure to scale headcount quickly, which means it hires in windows rather than on a fixed annual calendar. Those windows are exactly where a well-prepared outsider gets in. Chapter 16 covers how to find them before the job is posted.

Where the work is

Bengaluru holds roughly 34 per cent of the sector, Hyderabad about 14 per cent, with Chennai and Pune next. That concentration is real, and it is why the default advice is 'go to Bengaluru'. But it is no longer the whole picture, and Part Two explains why the fastest-growing hiring is happening elsewhere.

AI is not a chapter in this story, it is the plot

- Around 250,000 professionals in Indian GCCs now work in AI-related roles, and India is the largest AI talent hub outside the United States.
- More than 1,200 GCCs have already embedded AI and machine learning capability, supported by over 250 dedicated centres of excellence.
- Nearly half of all GCCs established since FY2021 were designed with AI as a core focus from day one, not retrofitted later.

For you, this has one practical consequence. The fresher who can work with AI systems - validate their output, wire them into a workflow, evaluate whether they are any good - is being hired. The fresher who can only write code that an AI now writes is being squeezed. Chapter 12 is entirely about this, and it is the highest-leverage chapter in the book for anyone graduating in the next two years.

> **WHAT THE NUMBERS DO NOT SAY**
> A $98.4 billion sector with 2.36 million people sounds like a wide open door. It is not. The sector is large and growing, but it is also selective, and the selection criteria changed recently. Size tells you the opportunity exists. It tells you nothing about whether you will be picked.

CHAPTER 3

The fresher squeeze, explained properly

In 2026 there are two true statements about the Indian fresher job market, and they point in opposite directions. Overall entry-level hiring is weak. GCC entry-level hiring is strong. If you only read one of those headlines, you will make a bad decision. So here is the full picture, with sources.

What contracted

According to TeamLease research reported by the Economic Times in May 2026, IT services fresher hiring fell about 11 per cent in FY26, from roughly 125,000 to 110,000. The reason given is specific and worth understanding: AI automated the L1 work - coding, testing, support - that historically absorbed freshers. The sector's revenue-to-headcount link is breaking. TCS, for instance, cut 23,460 jobs while growing revenue.

Bain data reported in October 2026 shows job postings in Q2 2026 down about 5 per cent year on year overall, but junior and entry-level postings down about 11 per cent. The contraction is sharper at the bottom of the ladder than in the market as a whole. A separate October 2026 analysis of 18,752 live tech postings found only 4.3 per cent tagged entry or junior level, against 51.5 per cent asking for senior candidates.

Set that against supply. Roughly 1.5 million students graduate from computer science and IT streams in India every year. When a market with 1.5 million annual entrants narrows its entry door, the competition per seat rises sharply. That is the real story behind every 'freshers cannot get jobs' post you have seen.

What expanded

In the same period, GCC fresher hiring rose to roughly 140,000 to 150,000 in FY26, helped by about 100 new centres opening and existing centres expanding. TeamLease projects GCC fresher intake to grow a further 12 per cent in FY27.

The foundit Insights Tracker for July 2026 puts the shift even more starkly: GCCs increased their share of fresher hiring from 8 per cent to 11 per cent, the biggest gain of any employer category, while the IT sector's share fell from 32 per cent to 24 per cent. Meanwhile overall technology hiring grew 10 per cent year on year. Demand did not disappear. It changed hands.

8% to 11%	32% to 24%	+10%	-11%
GCC SHARE	IT SHARE	TECH HIRING	ENTRY POSTINGS
of fresher hiring, 2025 to 2026	of fresher hiring, same period	year on year, July 2026	Q2 2026, Bain data

Why this happened, in one paragraph

A GCC exists to own a process end to end for a global parent. When that parent deploys AI, the work does not vanish - it moves up a level, and someone has to run, validate and improve the AI. That someone is often a graduate with the right foundations, because they are cheaper than a mid-level hire and more adaptable. At the same time, an IT services vendor whose revenue depends on billable hours has the opposite incentive: automate the billable junior work and stop hiring juniors. Two business models, two opposite reactions to the same technology. This is why the two headline numbers diverge.

THE UNCOMFORTABLE IMPLICATION

If you are preparing for the 2020 version of the fresher job market - aptitude test, group discussion, mass drive, training, deployment - you are preparing for the segment that is shrinking. The growing segment does not test you that way. It asks what you have built.

Where the entry-level AI jobs actually are

One finding deserves its own paragraph, because it is widely misreported. foundit's July 2026 data shows AI-led roles now account for about 4 per cent of fresher hiring, up from around 1 per cent a year earlier. But more than half of those - 51 per cent - are in AI operations: data analysis, prompt engineering, AI operations and testing. Not AI model development.

This is good news for a fresher, because AI operations roles are learnable in months, not years. You do not need a research background. You need to understand how a model fails, how to test it, how to structure a prompt that produces reliable output, and how to measure whether an output is good. Chapter 12 gives you the specific skills and the project that proves them.

CHAPTER 4

Four kinds of GCC, and which ones hire freshers

'Get a job at a GCC' is too vague to act on. GCCs differ enormously in what they do, and the difference determines whether they hire at entry level at all. There are four broad types, and they are not equally open to you.

Type 1: The delivery and operations centre

The original model. A global company moves a defined process - finance and accounting, customer operations, claims processing, IT support, data management - to India to run it at lower cost. The work is well-defined, measured against service levels, and process-driven.

Fresher friendliness: high. These centres hire in structured batches and train. Entry criteria are more about consistency and communication than about a

portfolio. If your college tier is modest and you have no projects, this is the most realistic entry point in the entire sector.

Trade-off: the work can be repetitive, and progression is slow and process-based. Use it as a door, not a destination. Two years in an operations centre with a self-built analytics portfolio is a very different position from two years in an operations centre with nothing.

Type 2: The engineering and product centre

The centre owns real software: a platform, a product, a set of services used by the parent's customers worldwide. This is where the highest entry-level pay sits, and where the interview is hardest. Expect an online assessment at LeetCode-medium difficulty, a project deep-dive, core computer science questions and a behavioural round.

Fresher friendliness: moderate, and entirely dependent on proof of work. These centres do hire freshers - often through graduate programmes and hackathons rather than open postings - but they filter hard on demonstrable capability. Chapter 9 and Chapter 10 are written for this door.

Type 3: The transformation and analytics hub

The centre exists to change how the parent operates: data platforms, analytics, automation, AI deployment, process re-engineering. The FY2026 landscape data shows this is where a lot of the growth is, and it is the type most likely to have an AI centre of excellence.

Fresher friendliness: moderate to high, and rising. These teams need people who can write SQL, build a dashboard, structure a business question and communicate a finding. The bar is a working understanding rather than deep specialisation, which makes this the best target for a non-CS graduate who has taught themselves data skills.

Type 4: The centre of excellence

A small, senior-heavy unit focused on a specialised capability - machine learning research, quantitative modelling, security research, silicon design, advanced engineering. High prestige, small headcount.

Fresher friendliness: low, with one exception. A 2026 Zinnov survey found that around 60 per cent of GCCs prefer experienced hires over fresh graduates for AI,

cloud and cybersecurity roles, citing the need for immediate expertise. CoEs are the sharpest expression of that preference. The exception is a research-track entry, usually via a thesis, a publication or a competition win - which is a different book.

> **HOW TO TELL WHICH ONE YOU ARE LOOKING AT**
>
> Open the parent company's careers page and read three of its India job descriptions. If the titles are process-oriented (Analyst, Associate, Process Specialist, Operations), it is Type 1. If they are product-oriented (Engineer, Developer, SDE, Platform), Type 2. If they mention data, analytics, automation or AI in the title, Type 3. If they ask for a PhD or a publication record, Type 4. Ten minutes of this tells you more than any ranking list.

The sequencing decision

If you can get into a Type 2 or Type 3 centre directly, do that. If you cannot, Type 1 is not a consolation prize - it is a legitimate two-year apprenticeship that puts a global brand on your CV, gives you enterprise exposure and pays better than most alternatives. The mistake is treating it as the end of the plan instead of the first stage of one.

The other legitimate sequence, which more people should consider: two to three years at an IT services company, then lateral into a GCC. TeamLease data from 2026 puts lateral movement from IT services into GCCs at 23 per cent - it is a well-worn path, not an unusual one. You get breadth and client exposure early, then depth and pay later. That is a slower route to the same place, and it is a real option if your college does not open GCC doors directly.

PART TWO

Choosing Your Lane

A GCC is not one job market. It is seven, with different skills, different interview formats and very different pay. Picking the wrong lane is the most common expensive mistake - people spend four months preparing for a door they were never going to fit through. These four chapters fix the lane before you spend any more time.

CHAPTER 5

The seven entry lanes

These are the routes through which a fresh graduate actually enters an Indian GCC in 2026. For each, the honest question is not 'is this prestigious' but 'can I produce evidence of capability in this in eight to twelve weeks'. If the answer is no, pick a different lane.

Lane 1: Software engineering

The largest lane. Software Engineer is the single most-posted title in Indian tech hiring - around 20 per cent of all postings in an October 2026 analysis of 18,752 live roles. At a GCC this means building and maintaining the parent's applications and services.

What they test: data structures and algorithms at a real problem-solving level, one language to interview depth, core computer science (databases, operating systems, networking basics), and your project. Your degree branch matters less than you think - what matters is that you can write correct code while explaining your reasoning.

Eight-week evidence plan: two medium-difficulty problems a day in your primary language, one full-stack project deployed with a live URL and a documented repository, and the ability to walk through every design decision in that project.

Lane 2: Data and analytics

Data Analyst is the second most-posted title nationally, around 13.5 per cent of postings, and Data Engineer around 7.7 per cent with the share nearly doubling between July and October 2026. This is the most accessible high-paying lane for people without a CS degree, because the entry bar is a toolset rather than a theory base.

What they test: SQL to a genuine intermediate level (joins, aggregation, window functions), Python for data work, one visualisation tool, and - critically - whether you can turn a messy question into a query and then explain the answer to a non-technical person.

Eight-week evidence plan: one end-to-end analysis on a real public dataset with a stated business question, a clean SQL script file, a dashboard, and a two-page write-up of what you found and what you would do about it. Chapter 10 gives the exact brief.

Lane 3: AI operations and applied AI

The fastest-growing lane and the most misunderstood. Remember the split: of fresher AI roles, 51 per cent are in AI operations - data analysis, prompt engineering, AI operations and testing - not model development. Titles you will see include AI Operations Analyst, AI Quality Analyst, Prompt Engineer, GenAI Associate, AI Solutions Associate.

What they test: whether you understand how a language model fails, how to design a prompt that produces reliable structured output, how to build an evaluation set and score outputs, and how to wire a model into a workflow. Very little deep mathematics.

Eight-week evidence plan: one small AI feature that works, with a written evaluation showing you tested it and measured it. Almost nobody at fresher level brings an evaluation set. Bringing one puts you in a different category. See Chapter 12.

Lane 4: Cloud and DevOps

DevOps is around 9 per cent of postings. Entry-level cloud roles at GCCs are real but narrower than bootcamps suggest, because cloud failures are expensive and teams prefer experience.

What they test: one cloud platform's fundamentals, Linux comfort, containers, CI/CD concepts, and increasingly a cost-awareness question ('this bill is too high, where would you look').

Eight-week evidence plan: one cloud certification at fundamentals level, plus a deployed project where the infrastructure is described in a README with a diagram and a cost estimate. The certification gets you screened in; the project gets you through the interview.

Lane 5: Engineering, research and design

A large and often-ignored lane. GCCs of automotive, aerospace, industrial, semiconductor and healthcare companies hire mechanical, electrical, electronics and instrumentation graduates into embedded systems, simulation, CAD, verification and digital manufacturing roles. Japanese GCCs alone - over 100 of them in India - skew heavily towards engineering-led industries.

What they test: domain fundamentals, one tool to depth (MATLAB, Simulink, ANSYS, Cadence, SolidWorks), and increasingly a scripting layer - Python or C for automation around the core engineering work.

Eight-week evidence plan: one simulation or design study with a written report in the format the industry uses, plus a Python script that automates part of your own workflow. The script is what separates you from a hundred identical CAD portfolios.

Lane 6: BFSI operations, risk and reporting

Banking, financial services and insurance GCCs are among the largest employers in the sector, and BFSI is one of the two sectors with the most untapped GCC expansion ahead. Entry roles include operations analyst, KYC and onboarding analyst, risk reporting analyst, fund accounting associate, regulatory reporting.

What they test: numerical accuracy, Excel to a high standard, attention to detail, regulatory awareness, and - heavily - communication, because these roles interface with global desks. A finance, commerce or economics degree is an advantage here, not a disadvantage.

Eight-week evidence plan: an Excel or SQL project on a real financial dataset, plus demonstrable knowledge of one domain area (derivatives basics, KYC requirements, IFRS fundamentals) that you can discuss for ten minutes without notes.

Lane 7: Business, product and functional

Business analyst, product associate, programme coordinator, marketing analytics, HR analytics, supply chain analyst. Product management roles inside GCCs have nearly doubled to about 20 per cent of the mix in five years, and some of that is entry-level adjacent.

What they test: structured thinking, case-style problem solving, stakeholder communication, and evidence that you understand the business domain. Technical skill is a bonus, not the requirement.

Eight-week evidence plan: one written product or process teardown - pick a real app or a real process, document how it works, identify three specific problems with evidence, propose fixes with trade-offs. This is a portfolio piece almost nobody has.

Lane	Best for	Hardest part	Evidence in 8 weeks
Software	CS/IT, strong problem solving	DSA under time pressure	Deployed app + repo
Data	Any degree, numerate	SQL depth beyond SELECT	End-to-end analysis
AI ops	Curious, fast learner	Proving rigour, not enthusiasm	Working feature + eval set
Cloud/DevOps	Systems-minded	Narrow entry-level demand	Cert + deployed infra
Engineering	Core branch graduates	Domain tool depth	Study report + script
BFSI ops	Commerce, finance, economics	Detail and compliance	Financial data project
Business	Any degree, strong writing	Structured thinking	Product teardown

HOW TO CHOOSE IN TEN MINUTES

Answer three questions. (1) Which lane can I produce a real artefact in within eight weeks, using what I already half-know? (2) Which lane has the most postings in the city I am willing to live in? (3) Which lane's daily work would I still find tolerable in year three? Pick the lane where all three answers are acceptable. Do not pick on prestige - prestige does not survive month fourteen.

CHAPTER 6

Reading a GCC job description like a recruiter

A job description is not a wish list. It is a filter specification, written by someone who has already decided what the shortlist looks like. Learning to read it backwards - from the filter to the evidence - is one of the highest-return skills in this book.

The frequency data: what Indian tech postings actually ask for

Two independent October 2026 analyses of live postings are worth internalising, because they tell you where to spend your study hours. Across 18,752 postings, Python appeared in 28.2 per cent and SQL in 21.6 per cent. Among cloud platforms, Azure led at 19.2 per cent, ahead of AWS at 17.1 per cent and Google Cloud at 8.5 per cent - a reversal of the global pattern that surprises many candidates.

A separate analysis of 289 entry-level-heavy postings found a different top of the list: communication skills in 36.0 per cent, ahead of Python at 35.3 per cent, then SQL at 20.1 per cent, Java at 18.7 per cent, JavaScript-family skills at 17.0 per cent, Git at 15.2 per cent, generative AI or LLM skills at 14.9 per cent and machine learning at 14.5 per cent.

28.2%	21.6%	36.0%	14.9%
PYTHON	SQL	COMMUNICATION	GENAI/LLM
of tech postings name it	second most requested	named as often as Python	already in fresher-adjacent JDs

Two conclusions follow. First, Python plus SQL plus clear communication covers the largest share of openings in the country. If you learn nothing else this year, learn those three. Second, generative AI is no longer a differentiator you can defer - it appears in roughly one in seven descriptions, at about the same rate as classical machine learning.

The four-line decoder

When you open a description, read it in this order.

1. **The title tells you the lane.** 'Analyst' in a bank GCC means reporting and operations. 'Engineer' means code. 'Associate' is usually a graduate programme. Do not apply to a title whose lane you cannot evidence.
2. **The 'must have' block is the actual filter.** Recruiters screen on the first three to five bullets. If you match four of five, apply. If you match two of five, the application is a lottery ticket - spend the hour on a better-matched role instead.
3. **The 'good to have' block is your differentiation list.** This is where a portfolio beats a CV. If a description says 'exposure to Snowflake or dbt is a plus', and you have built one pipeline in dbt, you have moved from 'maybe' to 'interesting'.
4. **The last paragraph tells you the real constraint.** Shift timings, location, 'flexibility to work across time zones', 'willing to relocate'. These are the reasons offers get withdrawn after acceptance. Read them before you invest.

Three phrases that mean something specific

- **'Skills-first' or 'no specific degree required'.** Around 15 per cent of fresher postings in 2026 specify no educational qualification at all. This is a genuine opening, not a typo. It means the assessment decides, and the assessment rewards preparation.
- **'Prior internship experience preferred'.** Roughly six in ten fresher postings now prefer or require internship exposure. If you have none, that is the gap to close first - see Chapter 17 for programmes, including a government scheme that explicitly includes GCCs.
- **'Own end-to-end'.** In 2026, 87 per cent of GCCs report owning end-to-end global processes. When a description says this, it means you will not be handing off to a client team. It also means the interview will test whether you can reason about the whole system, not one component.

What a description will not tell you

It will not tell you the salary - only about 3 per cent of Indian tech postings state one. It will not tell you the real work mode: in October 2026, just 0.8 per cent of live tech postings were fully remote, and a separate fresher-heavy sample had 84.8 per cent fully onsite. It will not tell you the team's health, which is the thing that determines whether you enjoy the job.

So use other channels for those three. Salary: Chapter 7 and a conversation with anyone two years ahead of you. Work mode: ask in the first call, before you invest in the process. Team health: ask 'what did this team ship in the last six months' and listen to whether the answer is specific.

> **THE COPY-PASTE TRAP**
>
> Do not send the same resume to forty postings. A recruiter sees the mismatch in four seconds. Ten tailored applications that mirror the description's own vocabulary will outperform forty generic ones - not marginally, but by a wide margin. Tailoring is not lying; it is translating what you have into the language the reader uses.

CHAPTER 7

Pay: what is real, what is noise

Money is where this market is most misreported. Screenshots circulate of Rs 21 lakh fresher packages, and they are true - for a very small number of specialised roles. The median is a different number, and you need both to make a good decision.

The headline comparison

TeamLease research reported by the Economic Times in May 2026 projected annual GCC entry-level salaries rising about 50 per cent to roughly Rs 7.5 lakh in the current fiscal year, while IT services fresher pay stays flat at about Rs 4.5 lakh. That is the single most useful sentence in this chapter. Same graduate, same city, roughly Rs 3 lakh a year difference, from employer type alone.

Two other data points frame it. The HAN Digital IT-BPM Graduate Hiring Outlook 2027 reports Tier-1 IT services players offering Rs 12 to 22 lakh for select specialised engineers in AI, data engineering, cybersecurity and enterprise tech,

while entry-level ITeS pay has sat at Rs 3 to 4 lakh for years. And the EY Future of Pay 2026 report puts GCC salary increments at 10.4 per cent for 2026, ahead of IT services at 9.6 per cent - so the gap is not closing, it is compounding.

Indicative entry bands by lane

The table below is a synthesis of published industry ranges for Bengaluru, 2026. Treat it as an order of magnitude, not a quote. Actual offers vary by company, city, role and your negotiation, and no honest source will give you a precise figure for a specific employer.

Lane	Indicative fresher band	What moves you up the band
Software engineering	Rs 6-10 lakh	Deployed projects, DSA strength, a strong internship
Data / analytics	Rs 6-10 lakh	SQL depth, a real analysis, business framing
Data science / ML	Rs 8-14 lakh	A deployed model, evaluation rigour, statistics
Applied AI / GenAI	Rs 10-18 lakh	Working AI feature plus an evaluation set
Cloud / DevOps	Rs 7-12 lakh	Certification plus deployed infrastructure
Core engineering (ER&D;)	Rs 6-10 lakh	Domain tool depth plus automation scripting
BFSI operations / risk	Rs 5-9 lakh	Excel mastery, domain knowledge, communication
Business / product	Rs 6-10 lakh	Structured thinking, a written teardown

Indicative only. Sources: published industry ranges, 2026.

CTC is not your salary

This trips up almost every first-job holder, so be precise about it. Cost to Company is what the employer spends, not what reaches your bank account. A Rs 7.5 lakh CTC does not mean Rs 62,500 a month.

- **Fixed pay** is the part you can rely on: basic salary, allowances, and usually a statutory bonus.

- **Variable pay** is a performance-linked component, often 5 to 15 per cent at entry level. It is not guaranteed. Ask what the actual payout was for the last two years, not what the maximum is.
- **Employer PF contribution** is part of CTC but is not in-hand. It is still your money, just later.
- **Gratuity** accrues after five years of service. At entry level, treat it as zero for planning.
- **Joining bonus, relocation, retention** are one-time. Strip them out before comparing two offers.
- **ESOPs and RSUs** now appear at individual-contributor level too - around 75 per cent of GCCs offer some form of equity or stock appreciation instrument beyond leadership, per EY's 2026 pay research. At entry level the value is usually modest and vests over years. Value it at a discount, and never at the headline.

The practical move: ask for the fixed-pay number, then compare only fixed pay across offers. If a recruiter will only discuss CTC, ask 'what is the fixed component, excluding variable and one-time payments' - it is a completely normal question and the answer is always available internally.

What is actually negotiable at entry level

Less than at mid-level, but not nothing. At fresher level, base pay is usually banded and immovable. What can move:

- **Your band placement**, if you have a competing offer or a specialised skill the team needs. This is the big one. A band is a range, and where you land in it is a decision someone makes.
- **Joining bonus**, if you are giving up another offer or a bonus at your current internship.
- **Relocation support**, if you are moving cities for the role.
- **Start date**, if you need to finish a degree or an internship properly.
- **Grade or title**, occasionally, if your experience genuinely maps to the next level.

What is not negotiable, so do not spend social capital on it: the variable percentage, the PF structure, the notice period, the bond if one exists. Ask about the bond before you accept - Chapter 26 covers how to read one.

THE ONLY COMPARISON THAT MATTERS

Compare fixed annual pay, in the city you will actually live in, after subtracting a realistic rent. A Rs 9 lakh offer in Bengaluru with Rs 30,000 rent is not better than a Rs 7.5 lakh offer in Coimbatore with Rs 12,000 rent. Do that arithmetic before you feel anything about a number.

CHAPTER 8

The city arbitrage

Everyone applies to Bengaluru. That is precisely why you should think carefully about whether you need to.

The concentration, stated plainly

Bengaluru holds about 34 per cent of India's GCC footprint, Hyderabad about 14 per cent, with Chennai and Pune next. In fresher hiring terms, the foundit data for July 2026 puts Delhi NCR at 19 per cent, Bengaluru, Mumbai and Pune at 14 per cent each, Hyderabad at 11 per cent and Chennai at 9 per cent. HAN Digital's 2027 outlook puts Bengaluru at about 25 per cent of onboarding, Delhi-NCR 17 per cent, Hyderabad 16 per cent, Chennai 15 per cent, Mumbai and Pune 10 per cent each.

So the metros are where the volume is. But volume is not the same as odds. Every one of those seats is contested by the same enormous applicant pool, and the applicant-to-role ratio in Delhi NCR has been measured at around 6.6 candidates per live role against 0.6 in Bengaluru. Concentration cuts both ways.

The part almost nobody acts on

Emerging cities already host more than 220 GCC units and are growing at roughly 11 per cent CAGR. ANSR's 2026 analysis found GCC job openings growing about 42 per cent in emerging cities against 19 per cent in metros. foundit data cited by HCLTech puts tier-2 and emerging cities at 15 per cent of GCC hiring, growing about 23 per cent year on year - nearly twice the metro pace.

Two more numbers make the case sharper. Lateral attrition in tier-2 GCC locations runs 8 to 12 per cent, against 18 to 22 per cent in tier-1. And JLL

estimates organisations can cut real estate, talent and operational costs by 25 to 50 per cent in tier-2 cities, which is why the expansion is happening at all.

For a fresher, all of that translates into one sentence: fewer applicants per seat, in a centre that is actively trying to build a team.

The emerging-city map

City	What is being built there	Notes for a fresher
Coimbatore	Engineering and manufacturing GCCs; 300,000+ experienced professionals in reach	Strong for ER&D; and core engineering; +20% fresher hiring growth reported
Jaipur	Technology and digital services	Proactive state GCC policy; smaller applicant pool
Indore	Technology, digital services, analytics	Growing fresher activity; strong engineering feeder colleges
Kochi	Technology, lower operating cost base	Good infrastructure, lower cost of living
Bhubaneswar	IT services and GCC support	Low cost, improving digital infrastructure
Visakhapatnam	Newer GCC activity	Small base, fast growth
Ahmedabad / GIFT City	Finance and fintech; India's only IFSC-backed GCC destination	Relevant for BFSI lanes specifically
Lucknow, Nagpur, Mysuru, Mangalore, Thiruvananthapuram , Chandigarh, Surat	Mixed; state-policy driven	Worth checking for your specific parent company

One structural reason to look here now: between 2024 and early 2026, ten states either drafted or announced dedicated GCC policies with incentives, skilling support and ready infrastructure. Policy tends to precede hiring by a year or two. The window where a state is offering incentives and the applicant pool has not caught up is the window worth using.

How to decide

1. **If you have a strong portfolio and can relocate freely:** apply to metros and tier-2 in parallel. Metros give you more shots; tier-2 gives you better odds per shot. There is no reason to choose.
2. **If you have a modest portfolio and no projects yet:** tier-2 is not a compromise, it is a strategy. Your competition is smaller and the centre is more likely to train you.
3. **If you have family or financial constraints on relocation:** start with the GCCs already in your city or the nearest hub. A job you can actually take beats a better job you cannot.
4. **If you are in a BFSI lane:** look at GIFT City, Mumbai, Pune, Hyderabad and Chennai specifically, and check which banks have centres where.

ONE HONEST CAVEAT

Tier-2 GCC roles skew towards operations, analytics and engineering support rather than deep product engineering. If your goal is to be a product engineer at a top-tier tech GCC, the metro is still the primary market. Use tier-2 to enter the sector, not necessarily to build the whole career.

PART THREE

Proof of Work

This is the part that decides whether you get shortlisted. Not the resume, not the college, not the certification list. The thing you built and whether you can defend it. Five chapters: the rule, five blueprints, the SQL bar, the AI skills that actually get hired, and how to present all of it.

CHAPTER 9

The one-project rule

Here is a pattern that repeats in every GCC interview debrief. Two candidates. The first has eight certifications, five small projects on GitHub, and a resume full of technologies. The second has one project, deployed, documented, and understood to the last decision. The second gets the offer, nearly every time.

The reason is structural. A GCC interviewer's real question is not 'what do you know'. It is 'can I put you in front of a global stakeholder in six months without embarrassment'. One project you can explain for forty minutes is evidence of that. Eight shallow projects is evidence that you collect things.

What 'one project' has to contain

1. **A problem statement in one sentence.** Not 'a machine learning project'. Something like: 'Which three product categories should this store stop stocking, based on 18 months of transaction data.' If you cannot state the problem in a sentence, you do not have a project - you have an exercise.
2. **A real dataset, or a real user.** A Kaggle competition dataset is acceptable if you frame it as a business problem. Real data from a real context is better. Data you invented is worse than useless, because the first follow-up question will expose it.

3. **One non-obvious decision.** Every good project has a moment where you chose between two reasonable options. 'I used XGBoost instead of logistic regression because the classes were imbalanced and I needed probability calibration for the ranking use case.' That sentence is what interviewers are listening for.
4. **A measured outcome.** Accuracy is not an outcome. 'Reduced the manual review queue by 40 per cent on a held-out set' is. 'Cut a four-hour weekly report to six minutes' is. 'The model scored 0.87 AUC' is a number, not an outcome - but it is acceptable if you explain what the number means for the decision.
5. **A live link or a reproducible repository.** If someone cannot open it, it does not exist. A README with a screenshot, setup steps and a one-paragraph summary is the minimum.
6. **A known weakness you can name.** 'It falls over above 500 concurrent users' or 'it does not handle multilingual input'. Volunteers who name their own limitations get trusted. People who claim perfection get probed until they crack.

> **THE FORTY-MINUTE TEST**
> Before you call a project finished, explain it out loud to someone who is not technical, for forty minutes, without notes. If you run out at eleven minutes, it is not finished. If you cannot answer 'why did you do it that way' three levels deep, it is not finished. This test is free and it is more predictive than any mock interview.

Why five shallow projects actively hurt you

Because they change the interviewer's first question from 'tell me about your project' to 'which of these is your best one, and why are the other four thin'. You have handed them a reason to be sceptical. One project removes that question entirely and moves the conversation to depth, which is where you want to be.

The exception: two projects, if they are paired

There is one defensible way to have two. Make them a pair that tells a story: one that shows you can build, and one that shows you can measure. For example, an analytics project that produces a recommendation, and a small deployed tool that acts on it. Together they say 'I can find the answer and ship it', which is the exact shape of a transformation-hub GCC role.

What does not work: two projects in unrelated domains, or two projects that are both tutorials with the names changed.

The timeline that actually works

Eight weeks, part-time, is realistic for a genuinely defensible project if you are disciplined. Four weeks is enough for a data analysis. Twelve weeks is what a deployed AI feature with an evaluation set needs. Plan for the number your lane requires, and start earlier than feels necessary - because the last two weeks are always documentation and debugging, and everyone forgets to budget for them.

CHAPTER 10

Five project blueprints

These are not tutorials. Each is a brief with a problem, a deliverable and a difficulty rating, written so that the output looks like real work at a GCC rather than a course submission. Pick one that matches your lane. Do not do all five.

Blueprint A: The decision memo (data / analytics lane)

Problem. Pick a real public dataset with a genuine business question attached - municipal transport ridership, e-commerce order data, electricity load, hospital admissions, UPI transaction volumes, crop yields. State one decision that the data could inform.

Deliverable. A SQL script file that builds the analysis from raw tables, a dashboard with no more than six charts, and a two-page memo: the question, the finding, the recommendation, the caveat. The memo is the point. Anyone can build a dashboard; almost nobody writes the memo.

What makes it GCC-grade. The caveat section. Real analysts state what the data cannot tell you. Add a paragraph on what you would need to collect to answer the question properly.

Difficulty. Low to medium. Four weeks part-time.

Blueprint B: The pipeline with a schedule (data engineering lane)

Problem. Take two or three public data sources that update, and build a pipeline that ingests, cleans, validates and stores them, running on a schedule, with a visible failure mode.

Deliverable. A repository with a containerised pipeline, a scheduled run, a data quality check that fails loudly, and a short architecture diagram in the README. Add a cost estimate for running it.

What makes it GCC-grade. The failure path. Describe what happens when the source changes its schema. Interviewers who run data platforms will ask exactly this, and most candidates have never thought about it.

Difficulty. Medium. Six weeks part-time.

Blueprint C: The AI feature with an evaluation set (applied AI lane)

Problem. Build one small, genuinely useful AI feature on top of a document set or a workflow - a support-ticket triage tool, a contract clause extractor, a meeting-notes-to-actions converter, a resume-to-role matcher. Then prove it works.

Deliverable. A working demo with a live link, plus an evaluation set of at least 40 hand-labelled examples, a scoring script, and a results table showing where the feature succeeds and where it fails. Include the prompt versions you tried and why the final one won.

What makes it GCC-grade. The evaluation set. This is the single strongest artefact a fresher can bring to an AI-adjacent GCC role in 2026, because it demonstrates the exact discipline enterprises are hiring for - measuring whether the model is any good before shipping it. Chapter 12 goes deeper.

Difficulty. Medium. Six to eight weeks part-time.

Blueprint D: The deployed service (software engineering lane)

Problem. Build a small full-stack application that a real person would use once a week - a shift-swap board, a study-group scheduler, a local event finder, a bill splitter with history. Not a to-do app.

Deliverable. Live URL, repository, README with an architecture diagram, authentication that actually works, and a short note on how you would scale it to 10,000 users and what would break first.

What makes it GCC-grade. The scaling note and the tests. A handful of unit tests on the core logic, plus an honest paragraph about the first bottleneck, reads as professional. It also gives the interviewer something to push on that you have already thought about.

Difficulty. Medium. Six weeks part-time.

Blueprint E: The process teardown (business / product lane)

Problem. Pick one real product or one real internal process. Document how it works today, with evidence - screenshots, screens, timings, error messages, support threads. Then identify three specific failures and propose fixes with trade-offs.

Deliverable. A six to ten page document: current-state map, three problems with evidence, three proposals with effort and impact estimates, and a recommendation. Include a one-page summary at the front for someone with five minutes.

What makes it GCC-grade. The evidence. 'The onboarding flow is confusing' is an opinion. 'The onboarding flow asks for the same document twice, at step 2 and step 5, and 40 per cent of the reviews I read mention it' is a finding.

Difficulty. Low. Three weeks part-time, and it is the most under-supplied portfolio piece in the entire market.

> **THE ONE THING ALL FIVE SHARE**
>
> Each ends in a written artefact a stranger could read and act on. That is what separates portfolio work from coursework. At a GCC, 87 per cent of centres own end-to-end global processes, which means someone has to write the thing that explains the process. Be the person who can write it.

CHAPTER 11

SQL to the level they actually test

SQL is the second most-requested skill in Indian tech postings at 21.6 per cent, and in data engineering roles it appears in 45.2 per cent of descriptions - the highest of any skill. It is also the cheapest skill on this list to reach interview standard, which makes it the best return on study hours in the book.

The problem is that most candidates stop at the wrong level. They can write SELECT, WHERE and a basic GROUP BY, and they believe that is 'knowing SQL'. It is not. Here is the honest bar.

Level 1: what everyone has

SELECT, WHERE, ORDER BY, basic GROUP BY with COUNT and SUM, and a single-table query. This is the level of a first-year course. It will not pass a screening test.

Level 2: the actual interview bar

- **Joins, all four kinds**, and knowing when a LEFT JOIN plus a WHERE clause silently turns into an INNER JOIN. This one mistake fails more candidates than any other.
- **Aggregation with HAVING**, and the difference between filtering rows and filtering groups.
- **Subqueries**, in the SELECT list, in FROM, and in WHERE - including correlated subqueries, which are slow but appear in every test.
- **Common table expressions** (WITH clauses). Interviewers use these to see whether you can decompose a problem. Write every multi-step answer as a chain of CTEs; it reads as professional.
- **Window functions**: ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, and SUM OVER with a PARTITION BY. If you can do 'the second-highest value per category' and 'month-over-month change' without looking anything up, you are at the bar.
- **Date handling** in your target dialect - DATE_TRUNC, date arithmetic, time zones. This is where candidates stumble in real tests.

- **NULL semantics**, and why NULL = NULL is not true. Interviewers ask this to separate people who have used SQL from people who have read about it.

Level 3: what separates a strong candidate

- **Query performance reasoning.** You can explain why an index helps, what a full table scan costs, and why a function on an indexed column disables the index.
- **Deduplication logic.** Given duplicate rows with timestamps, keep the latest per key. This appears constantly in real work and in tests.
- **Slowly changing dimensions**, at least conceptually - how you track a customer's changing address over time without losing history.
- **Dialect awareness.** Knowing that Snowflake, BigQuery, Postgres and SQL Server differ on date functions and window syntax, and knowing how to look it up quickly.

The four problems to be able to solve cold

1. For each customer, their most recent order and the number of days since. (Window function plus date arithmetic.)
2. Month-over-month percentage change in revenue, with the first month as NULL rather than zero. (LAG plus NULL handling.)
3. The top three products by revenue within each category. (RANK partitioned by category, filtered outside the window.)
4. Customers who bought in January but not in February, and the revenue at risk. (Anti-join or NOT EXISTS, plus aggregation.)

If you can write all four from scratch, in twenty minutes each, without searching, you are ready for a data interview at a GCC. That is a two-week goal, not a two-year one. Practise on a real dataset, not on puzzle sites - the puzzles teach syntax, real data teaches the thinking.

> **THE HONESTY RULE ON YOUR RESUME**
>
> If you list SQL, expect to be asked to write it live, share your screen, and explain it. Listing a skill you cannot demonstrate under mild pressure is the fastest way to lose a room. List what you can do cold. For everything else, write 'working knowledge' and be ready to say what you have actually done with it.

CHAPTER 12

The AI-adjacent skills that get you shortlisted

This chapter is the highest-leverage one in the book for anyone graduating in the next two years, so read it carefully and ignore the hype in both directions.

Start with the actual data

Three facts, each sourced. One: AI-led roles account for about 4 per cent of fresher hiring in mid-2026, up from around 1 per cent a year earlier. Small, but quadrupling. Two: of those roles, 51 per cent are in AI operations - data analysis, prompt engineering, AI operations and testing - not model development. Three: generative AI or LLM skills already appear in about 14.9 per cent of fresher-adjacent postings, roughly as often as classical machine learning at 14.5 per cent.

Put together, that says something precise: the entry-level AI job is mostly about making AI systems work reliably, not about inventing them. And it is already mainstream enough to appear in one in seven descriptions.

The five skills that actually get hired

1. Structured output design

Getting a model to produce output in a fixed schema, reliably, every time. This means schema definition, constrained decoding where available, validation on the way out, and a retry strategy when validation fails. Anyone can ask a model for JSON. Very few freshers can show you a validator, a retry policy and a failure log.

What to build: a small extraction tool that turns unstructured text into a fixed schema, with validation and a visible failure count. Ten lines of schema, thirty lines of validation, and you are ahead of most applicants.

2. Evaluation

The skill that separates people who have used AI from people who can be trusted with it in an enterprise. Evaluation means: define what 'good' looks like, build a labelled set, score the system against it, and report the result honestly including the failures.

What to build: 40 to 100 hand-labelled examples for one task, a scoring script, and a table of results by category. Then run your system against it, change one thing, run it again, and record what improved and what regressed. That last step - showing a measured iteration - is the single most impressive thing a fresher can put in front of an AI team in 2026.

3. Retrieval and grounding

Connecting a model to a specific document set so it answers from those documents rather than from memory. The concepts are chunking, embeddings, similarity search, and citation. You do not need to build a vector database from scratch - but you do need to understand why retrieval fails, which is usually chunking or the query, not the model.

What to build: a question-answering tool over a set of 50 to 200 documents (a public policy corpus, a set of standards, a manual), where every answer cites the chunk it came from. The citation is the point - it is what makes the output checkable, and checkability is what enterprises buy.

4. Workflow wiring

Taking a model output and making something happen with it - writing to a sheet, filing a ticket, sending a message, updating a record. This is where AI work becomes business value, and where most AI projects die. A fresher who can wire a model to a real system, with error handling, is immediately useful.

What to build: one workflow that takes an input, processes it with a model, validates the output, and writes the result somewhere a human will see it - with a log and a fallback when the model returns something unusable.

5. Cost and latency awareness

Knowing that a longer prompt costs more, that caching repeated context changes the economics, that a smaller model is often good enough, and that latency is a product decision. Interviewers at GCCs ask about this because their teams are measured on it.

What to build: add a cost and latency table to your AI project. Tokens in, tokens out, cost per thousand requests, median response time. It takes an hour and it changes how you are perceived.

WHAT NOT TO DO

Do not put 'AI expert' on your resume because you have used a chatbot. Do not claim to have fine-tuned a model if you called an API. Do not present a demo you cannot explain when it fails. The fastest way to lose credibility in an AI interview is to be vague about what you actually did - and AI interviewers are unusually good at spotting it, because they have heard the vague version a hundred times.

The honest learning path

1. **Weeks 1-2:** build one structured-output tool with validation. No framework. Just an API call, a schema, a validator and a retry.
2. **Weeks 3-4:** build a 40-example evaluation set for that task by hand. Score the tool. Write down what fails and why.
3. **Weeks 5-6:** improve one thing based on the failures. Re-score. Record the before and after.
4. **Weeks 7-8:** add retrieval over a document set with citations, and add the cost and latency table.

That is eight weeks and it produces one artefact that answers the only question an AI-adjacent GCC hiring manager has: can this person be trusted to ship something that works and to tell me honestly when it does not.

CHAPTER 13

GitHub and portfolio hygiene

Your repository is read by two kinds of people, and they look for different things. A recruiter looks for existence and activity. An engineer looks for quality and honesty. Both take under ninety seconds. Here is how to survive both.

The ninety-second recruiter read

- **Your profile has a bio and a pinned project.** Three lines about what you build, and the one project pinned so it is the first thing visible.
- **Repositories are named in plain language.** 'retail-demand-forecast' not 'ml_project_final_v2'. The name is the only thing read before the click.

- **Each pinned repository has a description.** One line. This is the highest-value field on GitHub and most people leave it empty.
- **There is recent activity.** A green square from this month. Not because streaks matter, but because a repository untouched for eleven months reads as abandoned.

The ninety-second engineer read

- **The README opens with what it does, in two sentences, and a screenshot.** Not with a table of contents. Not with installation instructions. What it does, then how to run it.
- **There is a stated result.** A number, a screenshot of the output, a table of scores. Engineers scroll for evidence.
- **There is a 'limitations' section.** This is counter-intuitive but it is the strongest credibility signal in a fresher repository. Naming what does not work tells the reader you tested it.
- **The commit history is not one giant commit.** Ten to twenty commits with readable messages shows process. One commit called 'final' shows a copy-paste.
- **There is no committed secret.** No API keys, no tokens, no .env file. An exposed key in a public repository is an instant rejection at any security-conscious GCC, and it happens to freshers constantly.

The README template

Use this structure exactly. It takes twenty minutes and it is the difference between a repository that is skimmed and one that is read.

1. **One-line description** at the top, as a heading.
2. **What it does** - two or three sentences, in plain language, no jargon.
3. **Screenshot or sample output** - an image, or a code block showing input and output.
4. **How it works** - four to six bullet points describing the pipeline, not the code structure.
5. **Results** - the measured outcome, with the caveat next to it.
6. **How to run it** - three commands maximum. If it needs more, containerise it.
7. **What it does not do** - two or three honest limitations.

8 **What I would do next** - one or two specific improvements.

The link you actually send

Do not send a repository link in an application. Send a single link to a one-page summary - a short document or a simple page - that contains the problem, a screenshot, the result, the limitation, and links to the code and the live demo. Applications are read by people with ninety seconds. Give them the ninety-second version and let the engineers go deeper if they want to.

This is the same logic as the decision memo in Chapter 10: the artefact that gets you shortlisted is the one written for the reader's time budget, not the one that proves the most work.

PART FOUR

Getting Found

A good project that nobody sees is a hobby. These five chapters are about distribution: how GCC recruiters actually find people, how referrals work in practice, how to catch programmes before they close, and how to apply so that a human reads it. This is where most capable candidates lose, and it is entirely fixable.

CHAPTER 14

LinkedIn is a search index, not a CV

The single most important structural fact about GCC hiring, and the one that surprises freshers most: GCC talent teams search for candidates rather than waiting for applications. They have a role, they open a search, they filter by skill and location and graduation year, and they message the results. A significant share of GCC hires happen before a role is ever publicly posted.

That means your LinkedIn profile is not a document you send to someone. It is a record that has to rank in a search. Those are different jobs, and most profiles are built for the wrong one.

How the search works

A recruiter searching for a data analyst does not search 'data analyst'. They search skills. They filter by location, by years of experience, by whether you are open to work, and often by the specific tools in the job description. Your profile surfaces if your skills section contains the words in their query, and if you have the activity that signals you are alive.

Which produces the first rule: the words matter more than the prose. A beautifully written About section that never mentions SQL will not appear in a SQL search.

The five fields that decide whether you appear

1. **Headline.** Not 'Student at XYZ College'. Use the formula: what you are, what you do, what you want. For example: 'Final-year CS student | Python, SQL, AWS | Building data pipelines | Looking for GCC data roles in Bengaluru'. It contains five searchable terms and it answers the recruiter's question in one line.
2. **Skills section.** Fill it to the maximum with the exact names used in job descriptions - 'SQL', 'Python', 'Power BI', 'Azure Data Factory', 'PySpark', 'pandas', 'Git'. Not 'databases', not 'programming'. Recruiters filter on tool names, so use tool names.
3. **About section.** Four short paragraphs: what you are studying, what you have built (with a link), what you are targeting, and how to reach you. First person. No adjectives about passion.
4. **Experience and projects.** List your project as an entry with a title, a date range and three bullets in outcome language. A project listed under 'Projects' is invisible to most filters; a project listed as an experience entry is not.
5. **Open to work.** Turn it on, with the specific titles you want and the cities you will accept. It is a filter field, and it is free.

Outcome language, with examples

This is the highest-return rewriting exercise in the book. Every bullet should follow: what you built, with what, and what it achieved.

Before (responsibility)	After (outcome)
Built a machine learning model for final year project.	Built and deployed a churn-prediction model (Python, scikit-learn, FastAPI) scoring 0.87 AUC on held-out data; documented the architecture and shipped it to a public endpoint.
Worked on data analysis using SQL.	Wrote 40+ SQL queries with window functions to analyse 1.2 million transaction rows, identifying three product categories with negative contribution margin.
Made a dashboard in Power BI.	Built a six-chart Power BI dashboard replacing a four-hour weekly manual report; the automated refresh runs in under six minutes.

Before (responsibility)	After (outcome)
Learned AWS and cloud computing.	Deployed a containerised API on AWS ECS with a documented cost estimate of Rs 1,400 per month at 10,000 requests per day.

Activity: the part people skip

Recruiters notice whether a profile is alive. You do not need to post daily, and you should not post motivational content - it actively hurts you with technical readers. What works, in order of return:

- **Comment thoughtfully on posts by engineers at your target companies.** Two or three sentences that add something. This gets you seen by exactly the right people, and it is the cheapest form of networking that exists.
- **Post one short write-up per project.** What the problem was, what you built, what you learned. Five paragraphs. Link the repository.
- **Follow and set job alerts on every target company's page.** This is how you catch postings in the first 48 hours, which Chapter 18 explains is decisive.

> **WHAT TO REMOVE TODAY**
>
> Delete or de-emphasise: the 'seeking opportunities' banner as your only headline, the list of twelve online courses with no project, the photo that is a group picture, the About section written in third person, and any skill you cannot discuss for two minutes. A short honest profile outperforms a long inflated one, because the interview is where inflated profiles fail.

CHAPTER 15

The referral playbook

Referrals carry disproportionate weight at GCCs, for a reason that is easy to understand: the parent company's culture puts a premium on fit and trust, and an internal recommendation is a cheap way to reduce risk. One warm introduction moves a profile from unread to shortlisted faster than almost anything else available to you.

Most people never ask, because they think they need a close contact. You do not. You need one person who works there, one specific role, and one message that is easy to act on.

Who to ask, in order

1. **Alumni from your college** at the target company. Use the company page, then the People tab, then search your college name. Highest response rate of any group, by a wide margin.
2. **People in the same role, one to three years ahead of you.** They remember their own search and are usually willing to help. Senior people are less responsive, not because they are unkind but because they get more messages.
3. **People who post about the company's work.** If someone wrote about a project their team shipped, they have already shown they engage publicly.
4. **The recruiter who posted the role.** Only after you have applied, and only with a short, specific note.

The first message

Keep it under ninety words. Ask for a conversation, not a referral - asking for a referral from a stranger in the first message is the most common mistake and it usually gets no reply.

> **TEMPLATE 1: FIRST CONTACT**
>
> Hi [Name] - I am a final-year [branch] student at [college], and I saw you moved into [team/role] at [company]. I have built [one specific thing: a dbt pipeline over municipal data / an evaluation set for a document-extraction tool] and I am trying to understand what [company]'s India team actually owns day to day before I apply to the [role] posting. Would you be open to a 15-minute call in the next couple of weeks? Happy to work around your calendar.

Why this works: it names the college connection, it proves you did something, it asks for a bounded amount of time, and it asks about the work rather than about a job. The referral ask comes later, once they have replied.

The conversation

Fifteen minutes. Have four questions ready and ask them in this order. They are designed to be interesting to answer, which is what makes people keep talking.

- 'What did your team ship in the last six months that you are proud of?'
- 'What does a new joiner actually do in the first ninety days?'
- 'What separates the people who do well here from the ones who struggle?'
- 'If you were me, with [your specific project], which of the open roles would you point at?'

Then send a thank-you note within a day, referencing one specific thing they said. That note is what makes you memorable three weeks later when a role opens.

The referral ask

Now, and only now, ask. Make it trivially easy for them to say yes.

> **TEMPLATE 2: THE REFERRAL ASK**
>
> Thanks again for the call - the point about [specific thing] changed how I am thinking about the role. I have applied to [exact role title] ([job ID or link]) and I think it is a genuine fit because [one sentence, one piece of evidence]. If you are comfortable referring me, I have attached a two-line summary and the link to my project so you do not have to write anything from scratch. If not, no problem at all - the conversation was genuinely useful.

Three things make this work. The role is specific and already applied to, so they are not doing your searching. There is a ready-made summary, so the effort is near zero. And there is an explicit easy out, which is what makes people actually respond rather than going quiet.

The follow-up that is not pestering

One follow-up, seven to ten days later, adding information rather than repeating the ask. 'I finished the evaluation set I mentioned - 40 examples, the tool now scores 82 per cent against it, write-up here. Still very interested in [role] if the team is still looking.' This is not a reminder. It is new evidence, which is why it is welcome.

WHAT NOT TO DO

Do not send a copy-pasted message to fifty people. Do not open with 'Sir/Ma'am, I hope this message finds you well'. Do not attach your resume to a first message. Do not ask 'any openings?' - it puts all the work on them. And never offer to pay anyone for a referral. That is how people get scammed, and Chapter 27 explains the pattern.

CHAPTER 16

The greenfield strategy

This is the least obvious tactic in the book and the one that most reliably produces interviews for candidates with modest profiles. It exploits a simple fact about how teams are built.

The insight

When a global company establishes a brand-new GCC in India, it needs to hire an entire team from scratch, against a headcount target and a timeline. During that window, the centre is more open to candidates it would normally filter out, because it has targets to hit and a culture to build. Once the centre matures, the filters tighten.

ANSR's 2026 analysis found that private-equity-backed companies accounted for 64 per cent of new GCCs established in emerging cities since 2020. PE-backed groups operate on value-creation timelines, which means they build fast, hire in windows, and are less attached to legacy hiring rituals.

How to find one early

- **Read commercial real estate news.** JLL, CBRE and Cushman and Wakefield publish office leasing reports. A multinational signing a large lease in Bengaluru, Hyderabad, Pune, Coimbatore or Jaipur is usually setting up or expanding a GCC. Lease news typically precedes public hiring by six to twelve months.
- **Watch state GCC policies.** Between 2024 and early 2026, ten states announced or drafted dedicated GCC policies with incentives and skilling support. Announcements name companies. Those names are your target list.

- **Follow the industry press.** NASSCOM's own publications, including the GCC Runway Monitor, identify which global enterprises are likely to establish centres next - its October 2026 edition found 517 Global Titans already present in India and about 1,199 still untapped, with the United States and Japan the largest sources.
- **Track the company's own careers page for a location that did not exist last quarter.** A new city appearing in the location filter is a signal.

What to do when you find one

Find the founding leadership - typically the India head, the head of engineering, and the HR lead - and reach out before the roles are posted. These people are building a team from nothing and are unusually responsive to a specific, evidence-backed message.

> **TEMPLATE 3: THE GREENFIELD MESSAGE**
>
> Hi [Name] - I read that [company] is opening its India centre in [city], focused on [what you can infer from the announcement or the parent's business]. I am a [year] graduate in [field] from [college], and I have spent the last two months building [specific thing with a link], because [the business reason]. I am not writing to ask whether there is a role posted - I know it is early. I am writing because I would like to be considered for the first [data / engineering / analytics] hires, and I would rather be in the conversation now than after the postings go up. I have attached a one-page summary. If it is useful, I would welcome fifteen minutes.

The phrase that makes this work is 'I know it is early'. It signals that you understand their situation, which almost no fresher does. And the specificity of the project gives them a reason to reply that has nothing to do with kindness.

The honest hit rate

Most of these messages get no reply. That is normal and it is not a verdict on you. Send eight to twelve, across different companies and different people, over three weeks. Two replies is a good outcome. One conversation that leads to a first-hire conversation is a very good outcome, and it happens often enough that the tactic is worth the hour it takes.

The reason it works better than applying is arithmetic: a posting gets hundreds of applications and one screening filter. A greenfield message reaches a person who is currently worried about filling seats.

CHAPTER 17

Programmes and hackathons: the real calendar

A meaningful share of GCC fresher and internship hiring runs through structured programmes and competitions rather than open postings. These are the highest-value channels available to a student, because the assessment is skills-based and college tier is often explicitly not a filter.

READ THIS BEFORE YOU PLAN ANYTHING

Programme names, windows and eligibility rules change every cycle. The table below tells you what kinds of programmes exist and where the front door is. It is not a calendar. Before you spend a single hour preparing for a specific programme, open the organiser's own page and confirm the current year's dates, eligibility and process. Third-party lists - including this book - go stale; the organiser's page does not. Never pay anyone for access to a programme.

The categories

Category	What it is	Where the front door is
Coding-challenge internships	An online assessment, top scorers invited to a paid summer internship, strong performers considered for a full-time offer. Stipends at the top programmes are substantial.	The company's own careers site and its page on competition platforms such as Unstop
Hackathons with hiring intent	A 24-48 hour build event run by the GCC itself, used as a primary fresher pipeline. Some include an assessment round before the event.	The company's careers site, under its social-impact or campus programmes
Graduate and associate programmes	A yearly structured intake with training, usually announced months ahead	Company careers site, filtered by city and 'graduate' or 'associate'
Government internship schemes	A paid internship placed with participating companies, including GCCs	The official scheme portal, pminternship.mca.gov.in

Category	What it is	Where the front door is
Campus and pooled drives	The GCC visits selected colleges, or runs a multi-college pooled process	Your placement office; ask which GCCs came in the last two years
Open competition platforms	Company-hosted challenges with interview or internship outcomes for strong performers	The company's page on the platform hosting the challenge

The government scheme worth knowing about

The Prime Minister Internship Scheme is a Government of India programme that places candidates in paid internships with participating companies. It matters here for one specific reason: Global Capability Centres are an explicitly eligible category of participating organisation under the revised guidelines, alongside selected MSMEs, SEZ companies, statutory authorities and state-nominated companies.

The current terms, from the Ministry of Corporate Affairs and PIB releases of 2026:

- Minimum financial assistance of Rs 9,000 per month, plus a one-time grant of Rs 6,000 for incidentals.
- Duration of 6 or 9 months, depending on the company's requirement.
- Age 18 to 25 as on the date of application, and not in full-time employment.
- Round III launched on 20 March 2026 under revised guidelines; 340 companies and organisations had participated as of 15 July 2026, across 32 states and union territories and over 380 districts.
- Since April 2026, final-year graduate and postgraduate students are also eligible, subject to a No Objection Certificate from the institution.
- Exclusions are specific and worth checking before you invest: graduates of IITs, IIMs, National Law Universities, IISER, NIDs, IIITs and IISc, and holders of professional qualifications such as CA, CMA, CS, MBBS, BDS, MBA or PhD are not eligible; nor is anyone with a family member in permanent government service, or family income above Rs 12 lakh in the preceding financial year.
- Applications go through the official portal only: pminternship.mca.gov.in.

For a candidate at a tier-2 or tier-3 college without access to corporate campus drives, this is a genuine route into a GCC-adjacent workplace with a stipend, and it is under-used relative to its scale.

How to prepare for a coding challenge, specifically

1. **Read the eligibility gate first.** Many challenges are restricted by graduation year, branch, CGPA or a college list. If you are not eligible, spending three weeks preparing is a pure loss. Check first, always.
2. **Practise the format, not just the topics.** Most of these assessments are timed, with two to three problems in 60 to 90 minutes, plus a multiple-choice fundamentals section. Practise under the clock, in the same interface if you can find a past paper.
3. **Do not neglect the MCQ section.** Candidates who only drill algorithm problems routinely fail on output-prediction questions about object-oriented concepts, operating systems and databases. Those are fast marks and they are frequently the difference.
4. **Solve the easy problem completely before touching the hard one.** Partial credit on three problems beats a perfect solution to one. This is the most common strategic error in timed assessments.
5. **Register early and screenshot the confirmation.** Eligibility and slots fill. Keep a record of what you registered for and when.

How to prepare for a hackathon

- **Pick the team before the problem.** Two people who ship beat four who debate. Ideally one who can build the interface and one who can build the logic.
- **Decide the demo in the first hour.** Work backwards from a three-minute demo. Everything that is not in the demo is not in scope.
- **Bring a working skeleton.** Authentication, deployment and a database connection set up in advance. Teams lose half their time to setup.
- **Write the submission before the last hour.** Judges read the write-up. A mediocre project with a clear write-up beats a good project with a rushed one.

THE COMPOUNDING EFFECT

One hackathon gives you an interview shot. Three give you a portfolio of problem statements you can talk about, a network of people who have seen you work under pressure, and the specific vocabulary that makes you sound like an insider. That compounding is the real prize, not the prize.

CHAPTER 18

Careers pages and ATS: applying so a human sees it

Applying is not the hard part. Being read is. Here is how the pipeline actually works and how to move your application from the pile to the shortlist.

The timing rule

Applications are reviewed in the order they arrive, and the strongest shortlists are often built in the first week. A posting that has been open for 25 days is usually already filled with a shortlist in progress. This single fact changes your strategy more than any resume tip: set alerts, apply in the first 48 hours, and treat late applications as a low-probability backup rather than a plan.

The channel hierarchy

1. **Referral plus application.** Best. The referral gets you read; the application gets you into the system.
2. **The company's own careers site.** Second best, and better than aggregators for a specific reason: postings there are current and complete, and third-party boards frequently list expired or duplicated roles.
3. **The company's LinkedIn page, with alerts on.** Good for speed. Apply through the company site even if you found it on LinkedIn.
4. **Job aggregators.** Useful for discovery and for seeing salary ranges, weak for conversion. Use them to build your target list, not as your main channel.
5. **Cold application with no referral and no contact.** The baseline. It works, at a low rate, and mostly in the first week.

What the ATS actually does

Applicant tracking systems are less magical than the internet suggests. They parse your document into fields, and they let a human filter and search. They do not usually auto-reject on keyword density. What they do is make your application findable or unfindable, and what humans do is search.

Practical rules that follow:

- **Submit a real text document, not an image.** A PDF exported from a design tool that turns text into outlines cannot be parsed. Check by selecting text in the PDF - if you cannot select it, neither can the system.
- **Use the job description's exact vocabulary.** If it says 'Power BI', write 'Power BI', not 'BI tools'. If it says 'PySpark', write 'PySpark'. Synonyms fail searches.
- **One column. No tables. No text boxes. No headers or footers holding contact details.** Parsers read left to right, top to bottom, and multi-column layouts scramble.
- **Standard section headings.** 'Experience', 'Education', 'Skills', 'Projects'. Creative headings like 'My Journey' break field mapping.
- **Put contact details in the body of page one.** Name, phone, email, city, LinkedIn URL, GitHub URL. Not in a header.
- **Name the file properly.** Firstname_Lastname_Role.pdf. Not resume_final_v3.pdf.

The resume shape that works for a fresher

One page. Always, at fresher level. Order matters, because a reader spends the first eight seconds on the top third.

1. **Contact block** - one line of links.
2. **One-line positioning statement** - the role you are targeting and the two skills that qualify you. This is not an objective statement; it is a filter aid.
3. **Projects** - two entries maximum, each with three outcome bullets and a link. Put this above education if your projects are stronger than your college brand, which is true for most people.
4. **Experience and internships** - including unpaid and freelance work. Frame it in outcomes.

5. **Skills** - grouped by category, using exact tool names, and honestly tiered: 'proficient', 'working knowledge'.
6. **Education** - degree, institution, year, CGPA if it is strong. If your CGPA is weak and the posting does not set a cut-off, omit it rather than highlight it - but never misstate it.
7. **Certifications** - only ones that are recognised and current. Three good ones beat eleven.

> **THE ONE THING THAT GETS APPLICATIONS DISCARDED**
>
> A resume that claims a skill the interview disproves. Recruiters talk to each other, and a candidate who listed 'advanced SQL' and could not write a window function gets remembered by name. Everything on the page is an invitation to be tested. Invite the tests you can pass.

PART FIVE

The Interview

Six chapters on the loop itself. What each round is testing, the questions you will be asked, and the specific answers that separate a shortlisted candidate from a hired one. Read Chapter 24 even if you read nothing else here - it is the round most freshers lose without realising they were being assessed.

CHAPTER 19

The loop, round by round

A GCC fresher process typically runs four rounds over two to six weeks, though it can compress to two weeks at a centre under headcount pressure. Each round exists to answer a different question, and the mistake candidates make is preparing for all of them in the same way.

Round 1: The online assessment

What it is testing: whether you can perform under time pressure, and whether your fundamentals are real. Not whether you are exceptional.

Typical shape: 60 to 90 minutes. Two or three coding problems at easy-to-medium difficulty, a multiple-choice section on computer science fundamentals, sometimes a SQL section, occasionally a short aptitude or reasoning block.

How people fail: by chasing the hard problem, by not reading constraints, by running out of time on the MCQs, and by not testing edge cases. Full details in Chapter 20.

Round 2: The technical interview

What it is testing: whether your project is real and whether you can reason out loud. This is where the offer is usually won or lost.

Typical shape: 45 to 60 minutes with one or two engineers. Roughly half the time on your project, half on fundamentals and a live problem. You will be asked to share your screen or write code in a shared editor.

How people fail: by not knowing their own project's decisions, by going silent while thinking, and by bluffing instead of saying 'I do not know, here is how I would find out'.

Round 3: The behavioural and stakeholder round

What it is testing: whether you can be put in front of a global stakeholder without embarrassment. GCCs weight this more heavily than IT services companies do, and freshers routinely underestimate it.

Typical shape: 30 to 45 minutes with a manager or a senior stakeholder. Structured questions about challenges, conflict, feedback, learning speed and working across cultures. Occasionally conducted by someone in another time zone, which is itself part of the test.

How people fail: by giving generic answers with no specifics, by criticising former colleagues, and by not being able to explain a technical project in plain language.

Round 4: The hiring manager round

What it is testing: fit with the specific mandate. Does this person want this job, in this team, doing this work?

Typical shape: 30 minutes, conversational. Career goals, why this company, why this team, questions about the role.

How people fail: by not knowing what the India centre actually owns. If you cannot say what the team builds, the interviewer concludes you applied to a brand rather than a job.

Round	Duration	Core question	Your preparation
Online assessment	60-90 min	Can you perform under time pressure?	Timed practice, MCQ fundamentals, easy-first strategy
Technical	45-60 min	Is the project real, can you reason?	Forty-minute project drill, live coding practice
Behavioural	30-45 min	Can we put you in front of a stakeholder?	Eight STAR stories, one project explained simply
Hiring manager	30 min	Do you want this specific job?	Know the team's mandate, three real questions

THE THING TO DO BETWEEN ROUNDS

After every round, write down every question you were asked, and the answer you gave. Within an hour. Then write the better answer. By round three you will have a document that is more useful than any interview guide, because it is specific to the company you are actually talking to.

CHAPTER 20

The online assessment

The assessment is a filter, not a ranking. Its job is to cut the applicant pool to something a human team can interview. That means the bar is usually 'competent and careful', not 'brilliant'. Which is good news, because competence and care are trainable in three weeks.

The coding section

- **Read every problem before starting any of them.** Ninety seconds spent triaging is the highest-return ninety seconds in the whole test. Solve in ascending order of difficulty, not in the order presented.
- **Write the brute force first, then optimise.** A working slow solution scores. An unfinished elegant one does not. Submit something that runs, then improve it if time allows.

- **Check the constraints before choosing an approach.** If the input size is 10 to the 5, an O(n squared) solution will time out and you have wasted the problem. If it is 100, it will not. This one habit saves more marks than any algorithm you can memorise.
- **Test the edges explicitly.** Empty input, single element, all identical values, maximum size, negative numbers. Most hidden test cases are edges.
- **Budget your time in thirds.** In 90 minutes with three problems: 30 minutes each, and if you finish early, bank the time rather than spending it perfecting one answer.

The MCQ fundamentals section

This is where strong coders lose offers, because they treat it as filler. It usually covers object-oriented concepts, database normalisation and joins, operating system basics (processes, threads, memory, deadlock), networking basics (what happens when you type a URL), and output-prediction questions in your language of choice.

Two hours of revision recovers most of these marks. Specifically: the four pillars of OOP with one example each, normal forms up to 3NF, the difference between a process and a thread, what a deadlock requires, and the DNS-to-HTTP request path. That is a short list and it appears constantly.

The SQL section

Usually two or three queries. Expect a join with aggregation, a grouped filter, and something requiring a window function or a subquery. Chapter 11 covers the bar. The most common failure is a LEFT JOIN that silently becomes an inner join because a WHERE clause filters on the right-hand table.

The aptitude block

Where present, it is usually quantitative reasoning, data interpretation and logical sequencing, 15 to 20 questions in 20 minutes. Do not skip it. Practice two sets the week before, purely to get the pace right - the questions are easy, the clock is not.

The three-week preparation plan

1. **Week 1:** 40 easy and 25 medium problems in your primary language, timed, in blocks of 45 minutes. One MCQ fundamentals set every two days.

2. **Week 2:** 25 medium and 5 hard problems. Two full-length mock assessments under real time conditions. Review every wrong answer and write down the reason.
3. **Week 3:** Two more full mocks. Revise your own error log rather than new material. Sleep properly before the real thing - fatigue costs more marks than ignorance in a timed test.

> **PRACTICAL RULES FOR THE DAY**
>
> Test your camera, microphone and browser an hour before. Close every other application - some platforms flag tab switching. Keep a pen and paper for working, since most platforms allow it. Read the instructions page for whether you can revisit questions. And if the platform crashes, screenshot the error and email the recruiter immediately; assessments fail, and the record matters.

CHAPTER 21

The project deep-dive: six questions you will always get

Every technical interview at every GCC will spend significant time on the thing you claim to have built. The questions are remarkably consistent. Learn the six, prepare a real answer to each, and this round becomes the easiest part of the process.

Question 1: 'Walk me through it.'

This is not an invitation to describe your file structure. It is an invitation to tell a story in four beats: the problem, the approach, the result, the limitation. Ninety seconds maximum, then stop and let them choose a thread.

A structure that works: 'The problem was X. I had Y data and needed to decide Z. I tried two approaches - A because it was simple, and B because it handled the imbalance better - and went with B after comparing them on a held-out set. It scored N, which means M for the decision. The main limitation is that it does not handle P, which is what I would do next.'

Question 2: 'Why did you choose that approach?'

The interviewer is checking whether you chose or copied. There is always a competing option, so name it and say why you rejected it. 'I used a simple rule-based classifier first, because it is interpretable and the team needs to explain decisions to a regulator. It was 12 points worse than the model, so I kept the model but added a reason-code output.' That answer demonstrates judgement, not just technique.

Question 3: 'What would you do differently?'

Have a real answer ready. Not 'nothing'. The strongest version names a specific technical decision you now regret, and why. 'I built it as a batch job. If I did it again I would stream the ingestion, because the source updates hourly and my daily refresh meant the dashboard was always a day stale.' This question is a gift - it lets you show self-awareness, which is the hardest thing to demonstrate otherwise.

Question 4: 'How would this scale?'

They are testing whether you think beyond your laptop. The honest answer for a fresher project is usually some version of: here is the first bottleneck, here is what breaks, here is what I would change. Name the specific limit. 'At around 500 concurrent users the single instance would queue. I would move the reads to a replica, cache the top queries, and only then consider sharding.'

Question 5: 'What does it not handle?'

Volunteer this before they find it. The candidates who say 'it fails on X and I know why' get trusted. The candidates who claim it works for everything get probed until they break, and then the interview is about their honesty rather than their skill.

Question 6: 'How did you test it?'

Most freshers have no answer to this and it is the fastest way to look junior. Even a handful of unit tests on the core logic, plus one paragraph on how you validated the output, puts you ahead. If you built an evaluation set - for an AI project - this is where you deploy it.

THE PREPARATION DRILL

Write the six answers out, in full, in a document. Then record yourself saying them, and listen back. You are listening for two things: whether you sound like you are reciting, and whether you say 'basically' or 'actually' as filler. Then do it with a friend who interrupts with follow-ups. The drill takes two hours and it is worth more than two weeks of additional study.

CHAPTER 22

Core computer science at GCC depth

GCC technical interviews test fundamentals more rigorously than IT services entry interviews, and more practically than product-company interviews. The topics are standard. The depth is 'can you reason about this', not 'can you recite a definition'.

The nine topics, and the level expected

Topic	What you must be able to do	Common question
Data structures	Choose the right one and justify it	Why a hash map here rather than a sorted array?
Complexity	Estimate time and space for your own code, out loud	What is the complexity of what you just wrote?
Databases	Normalisation to 3NF, joins, indexes, transactions and isolation	Why does this index not help this query?
Operating systems	Process vs thread, memory, scheduling, deadlock conditions	What happens when two threads update the same counter?
Networking	The request path, HTTP, DNS, status codes, latency	What happens between typing a URL and seeing the page?
OOP and design	Four pillars, composition over inheritance, when not to use OOP	How would you model this with classes?
Version control	Branching, merging, resolving a conflict, reading a diff	How would you handle a merge conflict in a shared file?

Topic	What you must be able to do	Common question
Concurrency basics	Race conditions, locks, why they are hard	How would you prevent duplicate processing?

The four answers worth memorising precisely

These four come up so often that a clean answer to each is worth the memorisation.

- **Process vs thread.** A process has its own memory space; a thread shares memory within a process. Threads are cheaper to create and switch, but they can corrupt shared state, which is why you need synchronisation. Processes are isolated, so one crashing does not take the other down.
- **What an index does.** It is a separate structure - usually a B-tree - that maps column values to row locations so the database can find rows without scanning the table. It speeds reads and slows writes, because every insert has to update the index too. Wrapping the column in a function usually disables it.
- **The request path.** DNS resolves the hostname to an IP, a TCP connection is established (and TLS negotiated for HTTPS), the browser sends an HTTP request, a load balancer routes it, the application queries a database or cache, and a response returns and is rendered. Latency lives in each hop, and the database is usually the slow one.
- **Deadlock conditions.** Mutual exclusion, hold and wait, no preemption, circular wait. Break any one and deadlock cannot occur - which is why lock ordering is the standard prevention.

The one behaviour that matters more than knowledge

Think out loud. An interviewer cannot assess reasoning they cannot hear, and a silent candidate looks stuck even when they are not. The pattern that reads as senior: state what you are considering, state what you are ruling out and why, then proceed. 'I could sort first, which would make this O(n log n) and then a single pass - but the array is nearly sorted already, so insertion sort would be close to linear. I will go with insertion sort and note the assumption.'

That paragraph contains no cleverness. It contains reasoning, and reasoning is what is being graded.

CHAPTER 23

The behavioural round: eight stories to prepare

The behavioural round is where GCCs differ most from IT services companies, and where freshers most often lose an offer they had otherwise earned. The reason is simple: a GCC works across time zones and cultures, so the interviewer is assessing whether you can operate in that environment. They do this by asking for evidence, not opinions.

The method

Use STAR - Situation, Task, Action, Result - but tighten it. Most candidates spend 70 per cent of the answer on the situation. Invert that: two sentences of situation, one of task, four of action, one of result. Ninety seconds total. Then stop.

The result must be specific. 'It went well' is not a result. 'We shipped a week early and the review queue dropped from 200 to 60 items a day' is.

The eight stories to have ready

1. **A time you learned something difficult quickly.** The most-asked GCC question, because the job changes constantly. Pick something with a measurable learning curve - a tool you had to learn in a week to finish a project.
2. **A time you disagreed with someone and what happened.** They are testing whether you can hold a position without becoming difficult. The best answers show you changed your mind on evidence, or changed theirs with evidence, or escalated cleanly.
3. **A time you received hard feedback.** Choose real criticism, not a humblebrag. Say what you did with it, and what changed afterwards.
4. **A time you failed.** A real failure with a real cost. Then the specific change you made so it would not recur. Never say 'I work too hard'.
5. **A time you worked with someone from a different background or time zone.** If you have not, say so honestly and describe the closest equivalent - a remote teammate, an online collaboration, a competition team from another college.

6. **A time you had to explain something technical to someone non-technical.** Prepare this one properly. It is the single most predictive question for GCC success.
7. **A time you took ownership of something outside your remit.** GCCs are flat, so initiative is visible and valued.
8. **A time you worked under a hard deadline with incomplete information.** Show how you decided what to cut.

The cross-cultural question, handled well

Many GCC roles involve regular contact with teams in Europe, North America or Southeast Asia. Interviewers weigh this heavily, and a general assurance that you are 'a good communicator' carries no weight at all. What carries weight is a specific instance.

If you have one, tell it concretely: who, what, what went wrong, how you fixed it. If you do not, do not invent one. Say plainly that you have not worked across time zones professionally, then describe what you have done that is adjacent - coordinating a remote project team, running an online competition, working with a client as a freelancer - and what you learned about written clarity from it. Honesty plus an adjacent example beats a fabricated story, and interviewers can tell the difference.

> **THE TEST BEHIND THE TEST**
>
> Every behavioural question at a GCC is ultimately asking one thing: if I put this person on a call with a stakeholder in London at 7 pm, would I worry? Answer that question with evidence, in every story, and the round takes care of itself.

CHAPTER 24

The ten-minute explanation test

If you take one skill from this book, take this one. It is the skill that gets freshers hired at GCCs, promoted in their first two years, and trusted with the work that matters. It is also the skill almost nobody practises deliberately.

The scenario

You have ten minutes with someone senior who is not technical and has three other meetings. They ask what you have been working on. You need to leave them with an accurate understanding, a reason to care, and confidence that you know what you are doing.

Most freshers respond with chronology: 'So first I collected the data, and then I had to clean it because there were nulls, and then I tried three models, and the first one did not work because...' Six minutes later, the listener knows nothing and has stopped listening.

The four-part structure

1. **The headline, in one sentence.** What you did and what it means. 'I found three product lines that are losing money on every sale, and I have a way to identify them automatically each month.'
2. **The stakes.** Why it matters to them. One sentence. 'It is about Rs 40 lakh a quarter across those lines, and nobody was tracking it because the reporting was aggregated at the wrong level.'
3. **The method, in one sentence and one caveat.** Not the steps - the logic. 'I built a margin calculation at SKU level and flagged anything negative after returns and shipping. The caveat is that the shipping cost is an allocation, so the ranking is directionally right rather than exact.'
4. **The ask or the next step.** What you want from them, or what happens next. 'I would like to review the flagged list with the category team before we act on it. Can I set that up for Thursday?'

That is ninety seconds. The remaining eight minutes are for their questions, which is exactly where you want the conversation - because their questions tell you what

they care about, and answering their questions demonstrates expertise far better than asserting it.

The three rules underneath

- **Lead with the answer, not the process.** This is the difference between a junior and a senior communicator, and it is learnable. Say the conclusion first. The method is the supporting evidence, not the story.
- **Quantify, then caveat.** A number with an honest limitation is more persuasive than a confident number without one. Senior listeners trust the person who names the caveat.
- **Never use a term they might not know without defining it in the same breath.** 'A window function - that is a way of comparing each row to others in its group - let me rank products within category.' The definition costs three seconds and buys credibility.

How to practise this week

Pick your project. Set a timer for ninety seconds. Explain it to a friend who does not work in technology, or to a family member. Ask them two questions: what did I build, and why does it matter? If they cannot answer both, your explanation failed, not their listening.

Do it four times, to four different people, and rewrite after each. By the fourth, you will have a version that works on anyone - and that version is what you will use in the interview, in the internship, and in every performance conversation for the next five years.

> **WHY THIS IS NOT SOFT SKILLS ADVICE**
>
> Communication is named as a requirement in 36 per cent of Indian tech job descriptions - as often as Python. It is not a nice-to-have that sits alongside the technical work. It is the mechanism by which technical work becomes visible, funded and trusted. A GCC owns end-to-end global processes; someone has to explain those processes to a parent company twelve time zones away. That someone is whoever can do it clearly.

PART SIX

Converting and Protecting

You have an offer, or an internship, or a link. Now the three chapters that determine what you actually get: how to convert an internship, how to read an offer letter properly, and how to recognise the scams that cluster around a market this desirable.

CHAPTER 25

Internship to offer: the conversion playbook

An internship is the highest-conversion door into a GCC. Industry estimates put internship-to-offer conversion at 40 to 70 per cent at leading GCCs - dramatically higher than the odds of a cold application. If you get one, treat it as a three-month interview, because that is exactly what it is.

Week 1: what actually matters

The first week is not about showing what you know. It is about removing every reason for someone to worry about you. Specifically:

- **Get access and tooling working on day one.** Nothing damages a first impression like being blocked on permissions for a week. Ask for everything you need in one message, on day one, listed.
- **Write down every acronym.** Every organisation has fifty. Keep a document. Asking what an acronym means in week six reads very differently from week one.
- **Find out what your manager is measured on.** Ask directly: 'What does success look like for this team this quarter?' The answer tells you where to spend your effort.

- **Meet three people outside your immediate team.** Deliberately, in week one. These are the people who will mention your name in a conversion discussion you will never attend.

Weeks 2 to 10: the work that gets noticed

Interns who convert do four things that interns who do not convert fail to do.

1. **They finish things.** A small completed task beats a large incomplete one, every time, because completion is the thing managers cannot do themselves. Scope down aggressively and ship.
2. **They write things down.** A one-page document that explains what you found, what you did and what you recommend is disproportionately visible in a flat organisation. Most interns produce code and no writing. Be the exception.
3. **They ask good questions at the right time.** Batch your questions. Ask them with your own attempted answer attached: 'I am seeing X. I think it is because Y, so I am going to try Z - does that sound right?' That is a colleague's question, not a student's.
4. **They make their manager's life easier without being asked.** Notice a report that is assembled manually and automate it. Notice a recurring question and write the answer down once. This is the single most reliable conversion behaviour there is.

Week 11: the conversation you must initiate

Most interns wait to be told. Do not. Around three weeks before the end, ask your manager for fifteen minutes and use this structure.

> **TEMPLATE: THE CONVERSION CONVERSATION**
> I want to make sure I use the last few weeks well, and I would also like to understand what happens next. Three things. First, here is what I think I have delivered: [two specific outcomes with numbers]. Is that how you would describe it? Second, where do you think I am still weakest, and what could I do in three weeks that would change your view? Third, I would like to be considered for a full-time role on this team. What is the process, and what would you need from me to support that?

Three moves in that message. You state your value in their terms. You ask for the criticism before they volunteer it, which is disarming and useful. And you ask

directly for what you want, with a question about process rather than a plea. Managers respect the directness, and the ones who were going to convert you anyway now have a reason to say so out loud.

The exit artefact

On your last day, leave behind a handover document: what you built, where it lives, how to run it, what is unfinished, and who to ask about what. Send it to your manager and your team. This document will be read again in three months when someone needs to change your code, and it will be the reason someone says your name when a role opens.

> **IF THE ANSWER IS NO**
>
> Sometimes the team has no headcount, or the conversion decision belongs to someone who has never met you. That is not a verdict on your work. Ask for two things before you leave: a written reference or a LinkedIn recommendation, and an introduction to one person in another team. Both are cheap for them and valuable for you. Then keep the relationship warm with one message every few months - the person who says no in March is often the person who says yes in September.

CHAPTER 26

Reading the offer

An offer letter is a contract written by a company's lawyers for the company's benefit. It is not dishonest, but it is not written to help you. Read it in this order.

The six clauses that matter

1. **Fixed pay breakdown.** Basic, allowances, and the ratio between them. Basic salary drives your PF and gratuity, so a low basic with high allowances is worth less than the headline suggests. Since the labour codes came into force, the requirement that basic pay be at least 50 per cent of total remuneration has been the reference point - check where your structure sits.
2. **Variable pay.** The percentage, the conditions, and - ask this - the actual payout for the last two years. 'Up to 15 per cent' and '15 per cent' are different offers.

3. **Bond or service agreement.** If there is one, note the amount, the duration, and precisely what triggers it. Training bonds are common in some segments. A bond is not automatically a reason to decline, but you must know the number before you sign.
4. **Notice period.** At entry level, 30 to 90 days is typical. Note whether there is a probation period with a shorter notice, and whether notice pay recovery is specified.
5. **Location and work mode.** Written, not verbal. 'Base location: Bengaluru' with a separate clause permitting relocation is different from a fixed location. If hybrid is promised, get the number of days in writing.
6. **Shift and allowance.** GCCs work across time zones, so shift allowances and night-shift transport are real terms. Ask whether the role requires fixed shifts or rotational, and what the allowance is.

The questions to ask before you sign

- What is the fixed annual component, excluding variable and one-time payments?
- What was the actual variable payout for the last two years, as a percentage?
- Is there a bond or service agreement? What is the amount and what triggers it?
- What is the notice period during probation, and after?
- Is the work mode hybrid, and if so how many days in office, and is that written into the letter?
- What is the shift pattern, and is there a shift allowance?
- When is the first increment cycle, and are freshers eligible in the first cycle?
- What is the joining date flexibility if my degree completion slips?

Comparing two offers honestly

Do this on paper, not in your head. Four columns: fixed pay, realistic variable, one-time payments, and cost of living. Then subtract a realistic rent and commute for the actual city. The offer that wins on paper is often not the one that feels bigger.

Item	Offer A (metro)	Offer B (tier-2)
Fixed annual pay	Rs 7,80,000	Rs 6,60,000
Realistic variable	Rs 60,000	Rs 50,000
One-time joining bonus	Rs 50,000	Rs 25,000
Annual rent (realistic)	Rs 3,60,000	Rs 1,44,000
Commute and living delta	Rs 60,000	Rs 20,000
Net after housing	Rs 4,70,000	Rs 5,71,000

Illustrative arithmetic. Use your own numbers.

That example is not an argument for tier-2 cities. It is an argument for doing the arithmetic. The metro offer may still be right - more roles later, more visibility, a denser network - but you should choose it for those reasons, knowingly, rather than because the headline number was larger.

THE ONE THING TO NEGOTIATE AT ENTRY LEVEL

Band placement, not base pay. Base pay is usually fixed by a grid. Where you sit inside the band is a decision a human makes, and it is influenced by a competing offer, a specialised skill the team needs, or a strong referral. Ask once, politely, with a reason. If the answer is no, accept gracefully - the relationship matters more than the increment.

CHAPTER 27

Scam defence

A market this desirable attracts predators. Freshers lose money, and sometimes more than money, to a small number of repeating patterns. Learn them once and you will never be caught.

The seven patterns

Pattern	How it looks	The tell
Paid placement	'We have an internal shortlist at [company]. Rs 15,000 to get your profile in.'	No legitimate employer or recruiter charges a candidate. Ever.
Fake recruiter	A Gmail or WhatsApp contact claiming to represent a known GCC, sending an offer letter as an attachment	The domain is not the company's. The offer letter has no employee ID, no HR signature block, no portal link.
Registration or test fee	'Pay Rs 500 to unlock the assessment link.'	Assessments are free. Always.
Training-then-job	'Complete our Rs 40,000 course and we guarantee placement.'	No one can guarantee placement. Ask for the written conversion rate.
Deposit or laptop fee	'Pay a refundable security deposit before joining.'	Employers do not take deposits from employees.
Document harvesting	A detailed form asking for Aadhaar, PAN, bank details and OTP before any interview	Personal documents come after an offer, through an official portal, never over chat.
Offer-letter fraud	A convincing letter with a salary that is implausibly high for the role, asking for an immediate joining fee	Verify by calling the company's published number, not the one on the letter.

The three-step verification that catches almost everything

1. **Check the domain, not the display name.** An email from 'careers@company-name.in' is not the same as from the company's actual corporate domain. Look at the characters carefully - substitutions of one letter are standard.
2. **Find the role on the company's own careers site.** If the job does not exist there and the company is large enough to have a careers site, treat the approach as unverified.
3. **Call the company's published switchboard or HR number, taken from the company's own website, and ask whether the person and the process are real.** Two minutes. It resolves nearly every case.

What to do if it has already happened

Report it, quickly, in three places: the cybercrime portal at cybercrime.gov.in, your bank if money moved, and the company being impersonated - their security or HR team wants to know, because their brand is being used. Keep every message and screenshot. Do not be embarrassed into silence; these operations are professional and they fool careful people.

> **THE PRINCIPLE, IN ONE LINE**
>
> You will never be asked to pay for a job. Not for a test, not for training, not for a deposit, not for a 'processing fee'. If money is flowing from you to them before you have a signed offer from a verified domain, it is a scam, and the details are decoration.

PART SEVEN

Execution

Everything above is knowledge. This part is the plan. A ninety-day schedule, a method for building your target list, and the trackers that keep it from dissolving into good intentions.

CHAPTER 28

The 90-day plan

Ninety days is enough to go from nothing to a credible candidate in a GCC process, if the work is sequenced correctly. The sequence matters more than the hours. Most people do these things in the wrong order - they apply first, then discover they have nothing to show, then panic-build, then apply again with a project they cannot defend.

Days 1 to 14: decide and set up

Day	Task	Output
1-2	Choose your lane from Chapter 5. Write one sentence stating it.	A written lane choice
3-4	Read 20 job descriptions in that lane and city. Note every recurring skill.	A skill frequency list of your own
5-7	Rewrite your LinkedIn headline, skills, About and one project entry (Ch 14).	A findable profile
8-10	Build a target list of 30 companies (Ch 29).	A spreadsheet with 30 rows
11-14	Rewrite your resume using outcome language (Ch 18). One page.	A resume that passes the eight-second test

Days 15 to 56: build the proof

Six weeks on one project, chosen from Chapter 10, matched to your lane. Non-negotiable rules: it gets deployed, it gets a README, it gets a written artefact, and it gets finished. Finishing is the discipline that separates the people who get hired from the people who nearly do.

- **Days 15-21:** scope. Write the problem statement in one sentence and the deliverable in three bullets. Cut anything not in those three bullets.
- **Days 22-42:** build the working version. Ugly is fine. Working is not optional.
- **Days 43-49:** add the thing that makes it professional - tests, an evaluation set, a cost table, whichever your lane requires.
- **Days 50-56:** write the artefact: README, decision memo, or teardown. Deploy. Then do the forty-minute drill from Chapter 9.

Days 57 to 76: distribute

- **Days 57-60:** set job alerts on all 30 target companies, and on LinkedIn for your lane plus your cities.
- **Days 61-70:** apply to every matching posting within 48 hours of it appearing. Ten to fifteen applications. Each one tailored to the description's vocabulary.
- **Days 63-76:** send 8 to 12 outreach messages a week - referral requests (Ch 15) and greenfield messages (Ch 16). Track every one.
- **Days 65-76:** check the programme and hackathon calendar twice a week (Ch 17). Register for anything you are eligible for.

Days 77 to 90: convert

- **Days 77-83:** assessment preparation, timed, three sessions a week (Ch 20).
- **Days 80-90:** write the six project answers (Ch 21) and the eight STAR stories (Ch 23). Record yourself. Do the ten-minute explanation drill (Ch 24) with four different people.
- **Ongoing:** every interview, within an hour, write down every question and a better answer. Review that document before the next one.

THE HONEST EXPECTATION

Ninety days of this does not guarantee an offer. It moves you from 'unlikely' to 'plausible', and it produces a portfolio that keeps working for you for two years. Most people who run this plan get their first interview inside eight weeks and their first offer inside five months. The people who fail are almost always the ones who skipped the six-week build and went straight to applying.

CHAPTER 29

Building your 30-company target list

A target list is not a list of famous companies. It is a list of specific centres, in specific cities, that hire your lane at entry level, with a named person to contact and a date to check. Build it once, properly, and it drives everything else.

The sourcing order

1. **Your own city and the two nearest hubs.** Filter the parent company's careers site by location and 'graduate', 'associate' or 'entry level'. Note whether the location has posted entry roles in the past year.
2. **The sector lists.** Work through the industries where GCCs concentrate: banking and financial services, insurance, retail and e-commerce, healthcare and pharma, automotive and industrial, technology and semiconductors, energy, and logistics. Two or three companies per sector gives you twenty-four.
3. **The expansion signals.** Commercial real estate reports, state GCC policy announcements and industry press tell you which companies are growing where. Add every one you find to the list with the date you found it.
4. **Your alumni network.** Search your college on the People tab of each target company. If nobody from your college works there, that is information too - it may mean the company does not recruit from your tier, or it may mean an opportunity nobody has used.

The columns that make the list useful

Column	Why it exists
Company and India centre type (Ch 4)	Tells you whether entry level is realistic
City and location filter checked	Confirms the centre actually hires where you will live
Lane fit (1-5)	Forces you to be honest about whether you match
Entry roles posted in last 12 months?	The single best predictor of whether they will again
Careers page URL	So you never search for it twice
Programme or hackathon they run	Your highest-probability entry route
One person to contact, with their role	An outreach message needs a name
Date of last contact	Prevents you from going quiet for six weeks
Status	Applied / referred / interviewing / rejected / dormant

The discipline that makes it work

Twenty minutes a day on the list, not a heroic weekend once. Specifically: five minutes checking alerts and applying to anything new, ten minutes on one outreach message, five minutes updating statuses. Twenty minutes a day for twelve weeks is fourteen hours of consistent, compounding effort, which beats a single frantic weekend by an enormous margin.

And prune. If a company has not posted an entry-level role in your lane in twelve months, move it to dormant and stop spending attention on it. A list of thirty live targets beats a list of sixty half-dead ones.

CHAPTER 30

The worksheets

Five worksheets to print or copy into a document. Fill them in by hand if you can - the act of writing makes the thinking real in a way that typing often does not.

Worksheet 1 - My lane, in one sentence

Worksheet 2 - The one project: problem, deliverable, three bullets

Worksheet 3 - Target list, first ten rows (company / city / lane fit / contact)

Worksheet 4 - The six project answers, in note form

Worksheet 5 - Eight STAR stories: one line each, then the one I will practise first

Appendix A - Where the GCCs are: a sector map

There is no complete public directory of GCCs in India, and any list you find online will be partial. What is useful instead is knowing which sectors concentrate them, so that you can go to a company's careers page and check for yourself. The NASSCOM-Zinnov FY2026 data counts 506 Forbes Global 2000 companies operating centres in India, alongside 583 mid-market and 504 private-equity-backed centres.

The table below is a research starting point, not an authority. It names sectors where GCC presence is well established and gives examples of parents that operate India centres. Use it to build your own target list in Chapter 29, and verify every entry on the company's own careers page before you spend time on it.

Sector	Examples of parents with India centres	Fresher lanes that fit
Banking and capital markets	JPMorgan Chase, Goldman Sachs, Morgan Stanley, Citi, Bank of America, HSBC, Barclays, Deutsche Bank, Standard Chartered, UBS, BNY, State Street, BlackRock, Fidelity	Software, data, BFSI operations and risk, business
Payments and cards	American Express, Mastercard, Visa, PayPal	Software, data, risk and fraud analytics

Sector	Examples of parents with India centres	Fresher lanes that fit
Insurance	Allianz, AXA, Swiss Re, Munich Re, AIG	Data, actuarial support, BFSI operations
Retail and e-commerce	Walmart, Target, Lowe's, Tesco, Costco, Amazon	Software, data engineering, supply chain analytics
Technology and semiconductors	Microsoft, Google, Adobe, Cisco, Qualcomm, Nvidia, Intel, Micron, Texas Instruments, Synopsys, Oracle, SAP, ServiceNow, Workday	Software, AI operations, cloud, verification
Industrial and manufacturing	Siemens, Bosch, Honeywell, GE, Schneider Electric, ABB	ER&D;, embedded, simulation, data
Aerospace and automotive	Airbus, Boeing, Mercedes-Benz, Volvo, Continental	ER&D;, embedded systems, digital manufacturing
Energy and utilities	Shell, BP, Schneider Electric	Data, engineering, digital operations
Healthcare and pharma	Optum, Elevance Health, Medtronic, Philips, AstraZeneca, Novartis, Pfizer, Sanofi	Data, regulatory reporting, engineering, AI operations
Consumer goods	Nestle, Unilever, Procter and Gamble, Coca-Cola, PepsiCo	Analytics, supply chain, finance operations
Logistics	Maersk, DHL, UPS, FedEx	Data, operations, optimisation
Japanese enterprises	Over 100 Japanese companies operate GCCs in India, concentrated in technology, industrials, automotive and healthcare	ER&D;, embedded, quality, digital manufacturing

Sector groupings for research only. Verify each employer's India presence and entry-level openings on its own careers page.

TWO SECTORS WORTH A SECOND LOOK

Energy and utilities, and BFSI, are the two sectors with the largest untapped GCC expansion ahead - NASSCOM's October 2026 GCC Runway Monitor puts white space at about 93 per cent in energy and utilities and 85 per cent in BFSI. White space means new centres. New centres mean greenfield hiring windows, which is the tactic in Chapter 16.

Appendix B - Forty questions you will actually be asked

These are the questions that recur across GCC fresher interviews, grouped by round. The right-hand column is what the interviewer is actually testing. Prepare an answer to each in note form, then say it out loud - reading your own notes silently creates a false sense of readiness.

Online assessment: what the paper is checking

Question form	What is really being tested
Two or three coding problems, timed	Whether you can produce working code under constraint, not brilliance
Output-prediction MCQs on your language	Whether your knowledge is real or memorised from tutorials
OOP, DBMS, OS and networking MCQs	Whether your fundamentals survived the degree
Two or three SQL queries	Whether you can join, aggregate and rank correctly
Quantitative and logical reasoning	Pace and accuracy on easy questions

Technical round: project and fundamentals

Question	What is really being tested
Walk me through your project.	Can you tell a story in four beats, or do you recite a file tree?
Why did you choose that approach?	Did you choose, or copy?
What did you try that did not work?	Do you iterate, and can you admit failure?
What would you do differently?	Self-awareness and technical maturity
How would this scale to a hundred thousand users?	Do you think beyond your laptop?
What does it not handle?	Honesty, and whether you tested it
How did you test it?	Whether you understand validation at all
What is the complexity of what you just wrote?	Whether you can reason about your own code
Why a hash map here rather than a sorted array?	Whether you choose data structures deliberately
What happens between typing a URL and seeing the page?	Networking fundamentals
What is the difference between a process and a thread?	Operating systems fundamentals
Why does this index not help this query?	Practical database knowledge
Write a query for the second-highest value per category.	Window functions
What happens when two threads update the same counter?	Concurrency awareness
How would you handle a merge conflict in a shared file?	Real collaboration experience
Tell me about a technology you learned recently.	Learning speed, and whether you can explain it
What would you do if you did not know the answer?	Whether you bluff

Behavioural round: the eight core stories

Question	What is really being tested
Tell me about a time you learned something difficult quickly.	Adaptability - the most-asked GCC question
Tell me about a disagreement with a colleague or teammate.	Whether you can hold a position without becoming difficult
Tell me about feedback that was hard to hear.	Coachability, and whether you changed anything
Tell me about a time you failed.	Honesty and whether you learned
Tell me about working with someone from a different background or time zone.	Global stakeholder readiness
Tell me about explaining something technical to a non-technical person.	The single most predictive question for GCC success
Tell me about taking ownership outside your remit.	Initiative in a flat organisation
Tell me about a deadline with incomplete information.	Prioritisation under pressure
Why do you want to work here?	Whether you know what the India centre actually owns
Where do you see yourself in three years?	Whether your plan is compatible with a flat hierarchy
Do you have any questions for me?	Whether you have thought about the job, not just the offer

THE ANSWER THAT LOSES THE OFFER

There is one answer pattern that fails consistently, and it is worth naming so you can avoid it: the answer with no specifics. 'I am a hard worker and a quick learner and I am passionate about technology.' Every interviewer has heard it four hundred times. Replace every adjective with an instance. Not 'I am a quick learner' but 'I had ten days to learn dbt for a project, and here is what I built on day seven.'

Appendix C - Glossary

Terms you will hear in GCC interviews and recruiting conversations, defined plainly.

- **GCC** - Global Capability Centre. A multinational's own India-based centre for technology, operations, analytics or engineering. Also called a captive or global in-house centre.
- **Captive** - the older term for a GCC, still used in real estate and policy contexts.
- **Parent company** - the global enterprise that owns the GCC.
- **Unit** - a distinct physical or organisational site of a GCC. India's 2,117 GCCs operate 3,728 units.
- **CoE** - Centre of Excellence. A specialised, senior-heavy unit within a GCC.
- **Greenfield** - a brand-new GCC being built from scratch. The best hiring window for an outsider.
- **FDE** - Forward Deployed Engineer. An engineer who works directly with enterprise customers to deploy and customise AI solutions. A role now appearing in GCC hiring.
- **ER&D;** - Engineering, Research and Design. The engineering-heavy GCC segment serving automotive, aerospace, industrial and semiconductor parents.
- **BFSI** - Banking, Financial Services and Insurance.
- **OA** - Online Assessment. The first-round test.
- **PPO** - Pre-Placement Offer. A full-time offer made at the end of an internship.
- **CTC** - Cost to Company. Total employer spend, not your in-hand salary.
- **Fixed pay** - the guaranteed component of your compensation. The only number worth comparing across offers.
- **Band** - the pay range attached to a grade. Your placement within it is negotiable; the band itself usually is not.
- **Lateral hire** - someone hired with experience rather than as a fresher. In 2026, around 23 per cent of GCC lateral hiring comes from IT services firms.
- **Skills-first hiring** - selection based on demonstrated capability rather than degree or college tier. Around 15 per cent of fresher postings now specify no educational qualification.

- **STAR** - Situation, Task, Action, Result. The structure for behavioural answers.
- **DPDP Act** - India's Digital Personal Data Protection Act. Relevant if you handle personal data in a GCC role.

Appendix D - The verification checklist

Before you act on anything in this book that has a date attached, run this checklist. It takes five minutes and it prevents the most expensive mistake in a job search: preparing for something that has already closed, or that you were never eligible for.

1. Is the programme or role on the organiser's own domain? If it only exists on a third-party list, it is unverified.
2. Is the current year's cycle open, and what is the exact closing date in the organiser's own words?
3. Am I eligible on every criterion - graduation year, branch, CGPA, age, college list?
4. What is the process, and how many rounds? Is there an assessment fee? (There should not be.)
5. What is the stipend or salary, and is it stated by the organiser rather than a blog?
6. Is there any payment required from me at any stage? If yes, stop.
7. Have I saved the confirmation email and screenshotted the application?

Appendix E - Sources

Every quantitative claim in this book traces to one of the following. Where a figure comes from an industry survey rather than an official release, that is noted in the text. All were current as of October 2026.

- Zinnov and NASSCOM, India GCC Landscape Report FY2026 (published May 2026, data as of March 2026) - GCC count, units, revenue, talent base, segment splits, growth since FY2021, AI talent and centre-of-excellence counts, functional role mix.
- NASSCOM, GCC Runway Monitor, October 2026 - 517 Global Titans present, 1,199 untapped, sector white space.

- EY, GCC Pulse Survey 2025 - 92 per cent beyond cost arbitrage, 87 per cent owning end-to-end global processes, 45 per cent in global decision-making, attrition, niche-skill hiring priorities.
- EY, Future of Pay 2026 - GCC increment projections of 10.4 per cent against IT services at 9.6 per cent, and equity participation rates.
- TeamLease research reported by The Economic Times, May 2026 - GCC fresher intake of 140,000 to 150,000 in FY26, +12 per cent FY27 projection, entry-level pay of about Rs 7.5 lakh against Rs 4.5 lakh in IT services, IT services fresher hiring decline of about 11 per cent, 23 per cent lateral flow, 1.5 million annual CS/IT graduates.
- HAN Digital, IT-BPM Graduate Hiring Outlook 2027, reported October 2026 - FY27 fresher hiring projection, employer-type shares, specialised salary bands against ITeS entry pay, city shares, tier-2/3 growth.
- foundit Insights Tracker, July 2026 - GCC share of fresher hiring rising from 8 to 11 per cent against IT's fall from 32 to 24 per cent, AI-led role share and the AI operations split, internship preference rates, qualification-free posting share, city shares, tech hiring growth.
- foundit data cited by HCLTech, September 2026 - tier-2 share of GCC hiring at 15 per cent growing 23 per cent year on year, attrition comparison.
- ANSR, India's Most GCC-Ready Emerging Cities 2026, reported May 2026 - 220+ emerging-city units, ~11 per cent CAGR, 42 per cent versus 19 per cent job-opening growth, 64 per cent PE-backed share of new emerging-city GCCs, the 14-city evaluation.
- JLL, cited by HCLTech, 2026 - 25 to 50 per cent cost advantage in tier-2 cities.
- Deloitte India, India's Strategic GCC Play for Japanese Enterprises, July 2026 - FY2030 economic impact projection, Japanese GCC count and sector mix, emerging-city list.
- Bain data reported by India Today, October 2026 - Q2 2026 posting decline and the sharper entry-level contraction.
- Careers360 analysis of 18,752 live postings, October 2026 - role shares, skill frequencies including Python at 28.2 per cent, SQL at 21.6 per cent, Azure at 19.2 per cent, AWS at 17.1 per cent, remote share at 0.8 per cent.

- Entry-level posting skill frequency analysis, October 2026 - communication at 36.0 per cent, Python at 35.3 per cent, generative AI at 14.9 per cent, salary disclosure at 3.1 per cent, onsite share.
- Ministry of Corporate Affairs and PIB releases, 2026, on the Prime Minister Internship Scheme - revised guidelines of 12 March 2026, Rs 9,000 monthly assistance and Rs 6,000 grant, 6 or 9 month duration, age and eligibility rules, GCCs as an eligible participating category, Round III launch on 20 March 2026 and participation figures.
- Zinnov survey, 2026, reported via CNA, October 2026 - around 60 per cent of GCCs preferring experienced hires for AI, cloud and cybersecurity roles.
- Company careers portals and programme pages for named employers - Walmart, JPMorgan Chase, Goldman Sachs and others - used only to identify that programmes exist, never for dates. Verify current terms on the organiser's page.

Where this book gives a range rather than a figure, it is because the published sources disagree and no official figure exists. Those ranges are labelled indicative. Where a claim could not be verified, it is not in the book.

A note on what this book is not

It is not a promise. No book can guarantee an offer, and anyone who tells you otherwise is selling something. What it can do is remove the two reasons capable people fail in this market: not knowing how the hiring actually works, and having nothing to show when it does.

Everything here is either verified from a primary or named industry source, or clearly labelled as a range where the published sources disagree. Where a date or eligibility rule is involved, the book tells you to check the organiser's own page before you spend time on it, because programmes move their windows every cycle and no third-party list stays current.

Read the market chapters once. Read the proof-of-work chapters twice. Then work the ninety-day plan, and let the plan do the rest.

Published by Digitalaikart | digitalkartai.shop
2026-27 Edition | October 2026